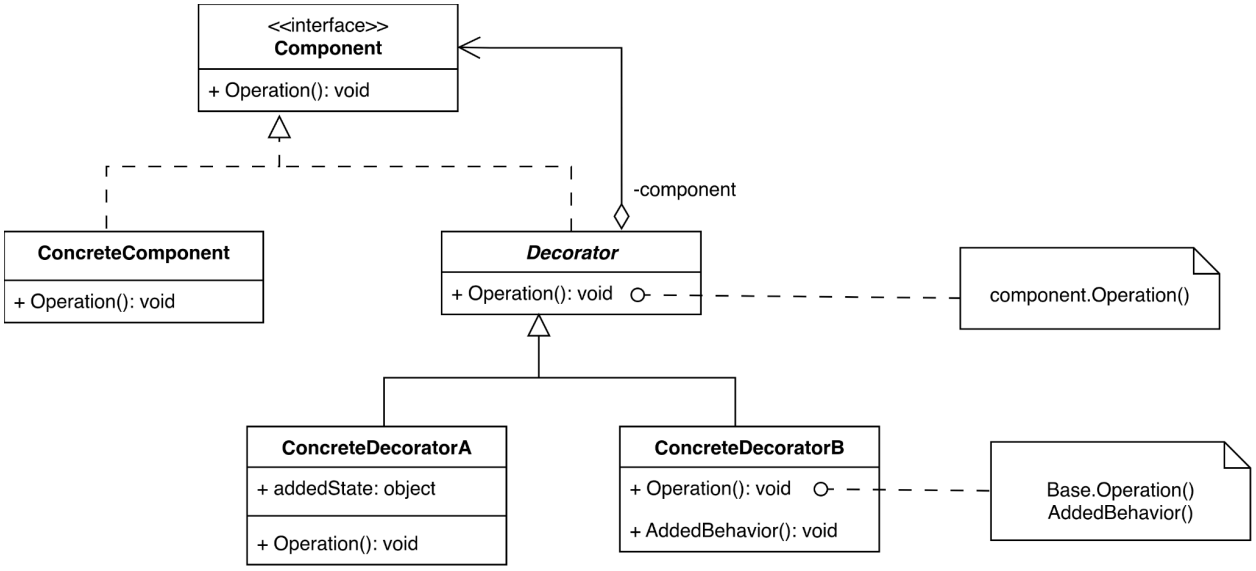
Decorator Pattern

Intent

ការបន្ថែមភារកិច្ច ឬមុខងារថ្មីៗទៅឱ្យ Object មួយដោយថាមវន្ត (Dynamically)។ Decorators ផ្តល់នូវជម្រើសដ៏មានភាពបត់បែនជាងការបង្កើត Subclass (Inheritance) សម្រាប់ការបន្ថែមសមត្ថភាពទៅលើមុខងារដែលមានស្រាប់។

UML Class Diagram



- Component
  - defines the interface for objects that can have responsibilities added to them dynamically.
- ConcreteComponent
  - defines an object to which additional responsibilities can be attached
- Decorator
  - maintains a reference to a Component object and defines an interface that conforms to Component's interface
- ConcreteDecorator
  - adds responsibilities to the component

Sample Code

```
using System;

// Component: The interface for objects that can have responsibilities added to
them.
public interface IComponent
{
    void Operation();
}

// ConcreteComponent: The original object we want to decorate.
public class ConcreteComponent : IComponent
{
    public void Operation()
    {
        Console.WriteLine("ConcreteComponent: Executing basic operation.");
    }
}

// Decorator: Maintains a reference to a Component object and defines
// an interface that conforms to Component's interface.
public abstract class Decorator : IComponent
{
    protected IComponent _component;

    public Decorator(IComponent component)
    {
        _component = component;
    }
```

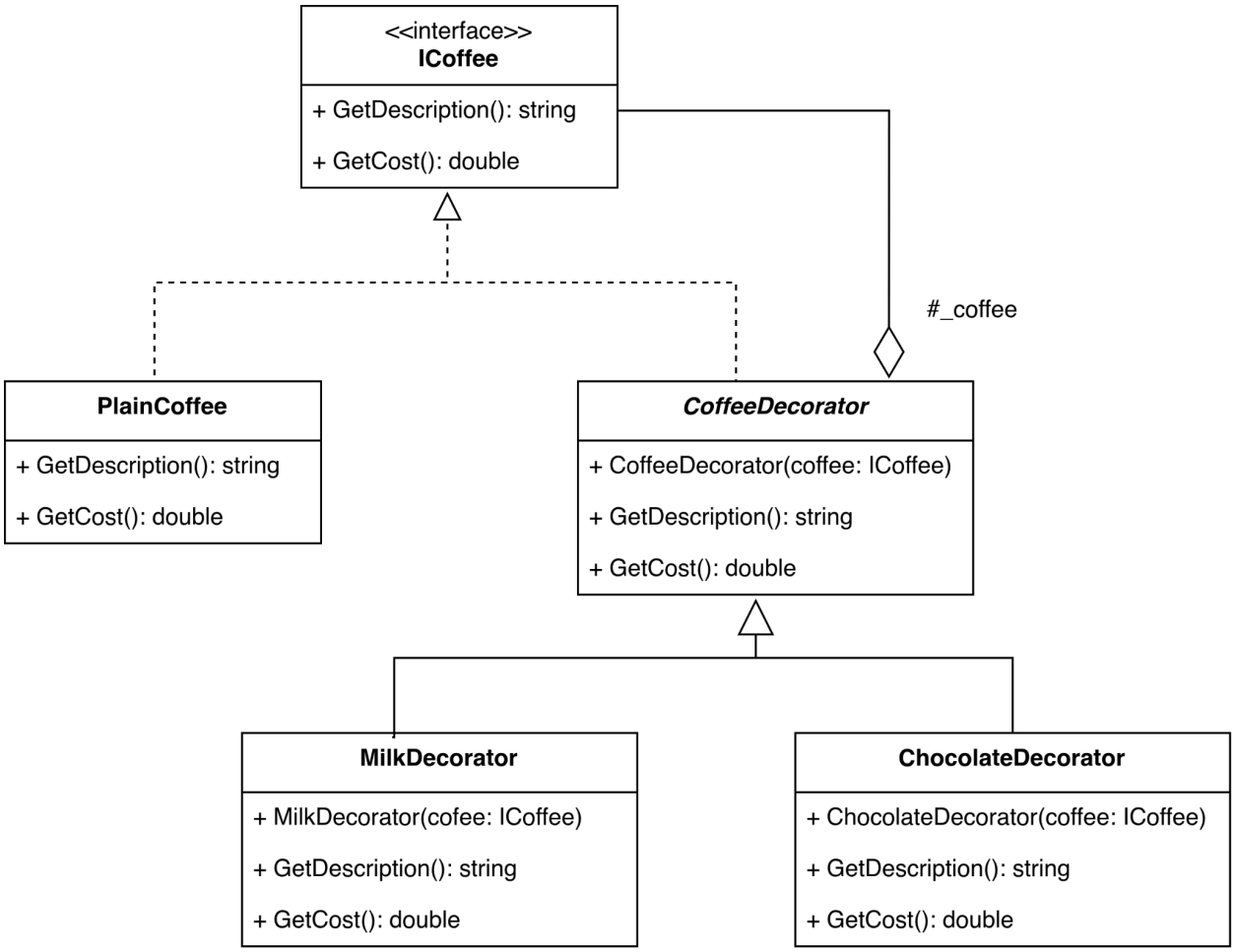
```
    public virtual void Operation()
    {
        if (_component != null)
        {
            _component.Operation();
        }
    }
}

// ConcreteDecoratorA: Adds state to the component.
public class ConcreteDecoratorA : Decorator
{
    private string _addedState = "Active";

    public ConcreteDecoratorA(IComponent component) : base(component) { }

    public override void Operation()
    {
        base.Operation();
        Console.WriteLine($"ConcreteDecoratorA: State is {_addedState}");
    }
}

// ConcreteDecoratorB: Adds new behavior to the component.
public class ConcreteDecoratorB : Decorator
{
    public ConcreteDecoratorB(IComponent component) : base(component) { }

    public override void Operation()
    {
        base.Operation();
```

```
        AddedBehavior();
    }

    void AddedBehavior()
    {
        Console.WriteLine("ConcreteDecoratorB: Executing extra behavior!");
    }
}

class Program
{
    static void Main()
    {
        // 1. Create a simple component
        IComponent component = new ConcreteComponent();

        // 2. Wrap it in Decorator A
        Decorator decoratorA = new ConcreteDecoratorA(component);

        // 3. Wrap Decorator A in Decorator B
        Decorator decoratorB = new ConcreteDecoratorB(decoratorA);

        Console.WriteLine("--- Starting Decorated Operation ---");
        decoratorB.Operation();

        // Output will show the base operation, then A's state, then B's behavior.
    }
}
```

Example

Coffee Shop

យើងមានកាហ្វេមូលដ្ឋាន (Basic Coffee)។ បើភ្ញៀវចង់ថែម ទឹកដោះគោ (Milk) ឬ គ្រាប់សូកូឡា (Chocolate) យើងមិនអាចបង្កើត Class ថ្មីសម្រាប់រាល់ការផ្សំ (Combination) ទាំងអស់នោះទេ (ព្រោះវាមានច្រើនពេក)។



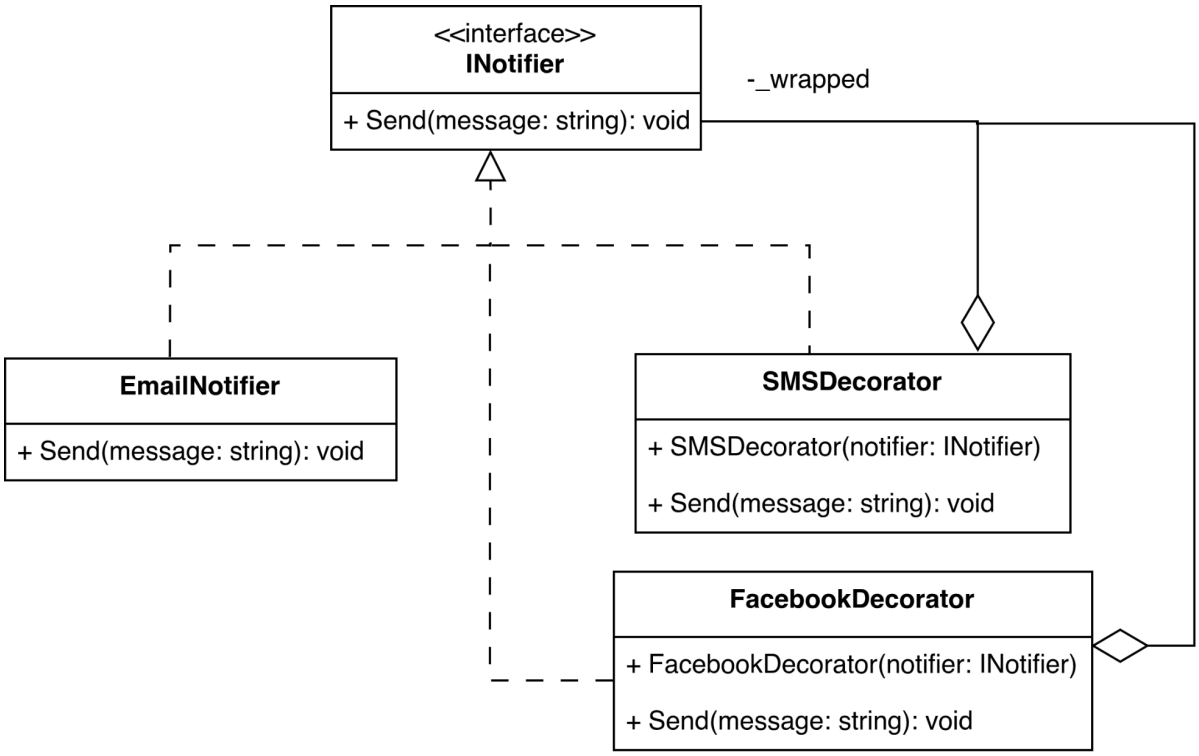

```
using System;

// 1. Component Interface
public interface ICoffee {
    string GetDescription();
    double GetCost();
```

```
}

// 2. Concrete Component
public class PlainCoffee : ICoffee {
    public string GetDescription() => "Simple Coffee";
    public double GetCost() => 2.0;
}

// 3. Base Decorator
public abstract class CoffeeDecorator : ICoffee {
    protected ICoffee _coffee;
    public CoffeeDecorator(ICoffee coffee) => _coffee = coffee;
    public virtual string GetDescription() => _coffee.GetDescription();
    public virtual double GetCost() => _coffee.GetCost();
}

// 4. Concrete Decorators
public class MilkDecorator : CoffeeDecorator {
    public MilkDecorator(ICoffee coffee) : base(coffee) {}
    public override string GetDescription() => base.GetDescription() + ", Milk";
    public override double GetCost() => base.GetCost() + 0.5;
}

public class ChocolateDecorator : CoffeeDecorator {
    public ChocolateDecorator(ICoffee coffee) : base(coffee) {}
    public override string GetDescription() => base.GetDescription() + ",
Chocolate";
    public override double GetCost() => base.GetCost() + 0.7;
}

// ការប្រើប្រាស់
```

```
// ICoffee myCoffee = new PlainCoffee();
// myCoffee = new MilkDecorator(myCoffee);
// myCoffee = new ChocolateDecorator(myCoffee); // កាហ្វេ + ទឹកដោះគោ + សូកូឡា
```

Notification System

កម្មវិធីរបស់អ្នកផ្ញើសារដំណឹងតាម Email ជាមូលដ្ឋាន។
ប៉ុន្តែអ្នកចង់ឱ្យអ្នកប្រើប្រាស់អាចជ្រើសរើសការផ្ញើបន្ថែមតាម SMS ឬ Facebook បានដោយសេរីនៅពេល Runtime។

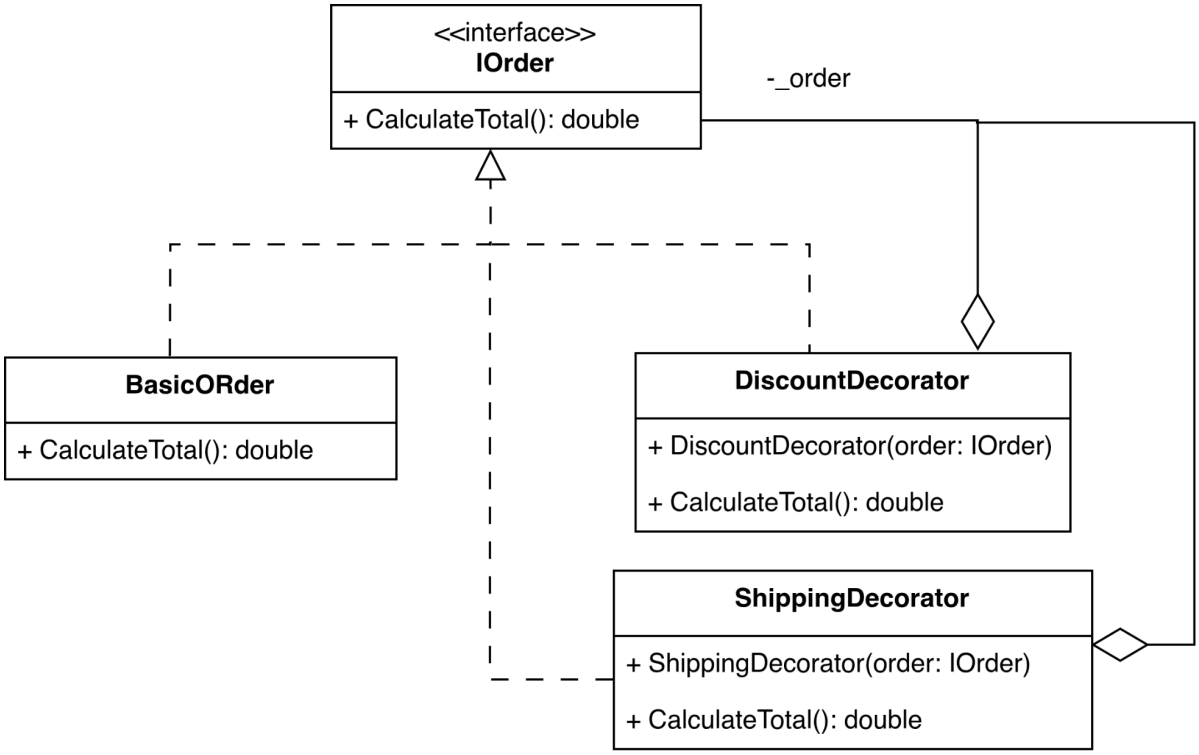



```
// 1. Component
public interface INotifier {
    void Send(string message);
}

// 2. Concrete Component
public class EmailNotifier : INotifier {
    public void Send(string message) => Console.WriteLine($"Sending Email:
```

```
{message}");
}

// 3. Concrete Decorators
public class SMSDecorator : INotifier {
    private INotifier _wrapped;
    public SMSDecorator(INotifier notifier) => _wrapped = notifier;
    public void Send(string message) {
        _wrapped.Send(message);
        Console.WriteLine($"Sending SMS: {message}");
    }
}

public class FacebookDecorator : INotifier {
    private INotifier _wrapped;
    public FacebookDecorator(INotifier notifier) => _wrapped = notifier;
    public void Send(string message) {
        _wrapped.Send(message);
        Console.WriteLine($"Sending Facebook Message: {message}");
    }
}
```

E-commerce Discount & Shipping

តម្លៃទំនិញអាចប្រែប្រួលអាស្រ័យលើការបញ្ចុះតម្លៃ (Discount) ឬសេវាដឹកជញ្ជូន (Shipping Fee)។ យើងត្រូវការបន្ថែមមុខងារទាំងនេះទៅលើតម្លៃដើមតាមស្ថានភាពជាក់ស្តែង។




```
// 1. Component
public interface IOrder {
    double CalculateTotal();
}

// 2. Concrete Component
public class BasicOrder : IOrder {
    public double CalculateTotal() => 100.0; // តម្លៃទំនិញដើម $100
}

// 3. Decorators
public class DiscountDecorator : IOrder {
    private IOrder _order;
```

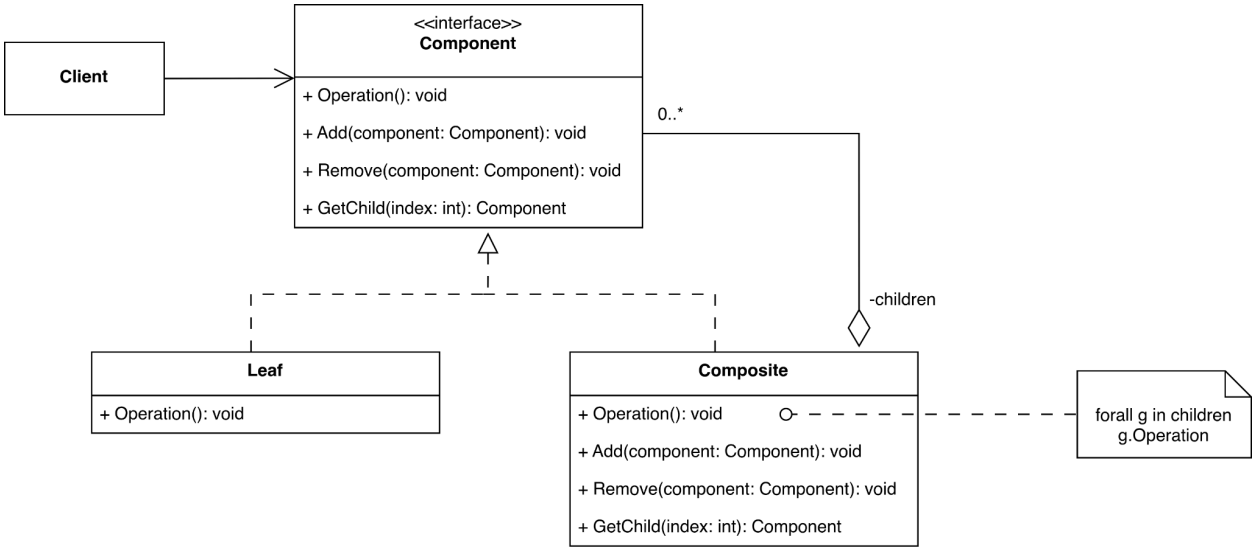
```
    public DiscountDecorator(IOrder order) => _order = order;
    public double CalculateTotal() => _order.CalculateTotal() - 10.0; // ចុះតម្លៃ $10
}

public class ShippingDecorator : IOrder {
    private IOrder _order;
    public ShippingDecorator(IOrder order) => _order = order;
    public double CalculateTotal() => _order.CalculateTotal() + 5.0; // ថែមថ្លៃដឹក $5
}
```

Composite Pattern

Intent

រៀបចំវត្ថុ (Objects) ឱ្យទៅជាទម្រង់មែកធាង (Tree structures)
ដើម្បីបង្ហាញពីហានានុក្រមនៃផ្នែកនីមួយៗដែលផ្គុំគ្នាជាសមាសធាតុរួម (Part-whole hierarchies)។
Composite អនុញ្ញាតឱ្យ Client ចាត់ទុកវត្ថុទោល (Individual objects)
និងវត្ថុដែលបង្កឡើងពីការរួមបញ្ចូលគ្នា (Compositions of objects) ក្នុងលក្ខណៈដូចគ្នាទាំងស្រុង។

UML Class Diagram



- Component
  - declares the interface for objects in the composition
  - implements default behavior for the interface common to all classes, as appropriate
  - declares an interface for accessing and managing its child components
  - (optional) defines an interface for accessing a component's parent in the
  - recursive structure, and implements it if that's appropriate.
- Leaf
  - represents leaf objects in the composition. A leaf has no children
- Composite
  - defines behavior for components having children
  - stores child components

        - implements child-related operations in the Component interface
    - Client
        - manipulates objects in the composition through the Component interface.

Sample Code

```
using System;
using System.Collections.Generic;

// Component interface
public interface IComponent
{
    void Operation();
    void Add(IComponent component);
    void Remove(IComponent component);
    IComponent GetChild(int index);
}

// Leaf
public class Leaf : IComponent
{
    private readonly string _name;

    public Leaf(string name) => _name = name;

    public void Operation() => Console.WriteLine($"Leaf [{_name}]: Operation()");

    public void Add(IComponent component) =>
        throw new NotSupportedException("Leaf cannot have children.");

    public void Remove(IComponent component) =>
        throw new NotSupportedException("Leaf cannot have children.");
```

```
    public IComponent GetChild(int index) =>
        throw new NotSupportedException("Leaf has no children.");
}

// Composite
public class Composite : IComponent
{
    private readonly string _name;
    private readonly List<IComponent> _children = new();

    public Composite(string name) => _name = name;

    public void Operation()
    {
        Console.WriteLine($"Composite [{_name}]: Operation()");
        // forall g in children => g.Operation()
        foreach (var child in _children)
            child.Operation();
    }

    public void Add(IComponent component) => _children.Add(component);

    public void Remove(IComponent component) =>
_children.Remove(component);

    public IComponent GetChild(int index) => _children[index];
}

// Client
public class Client
```

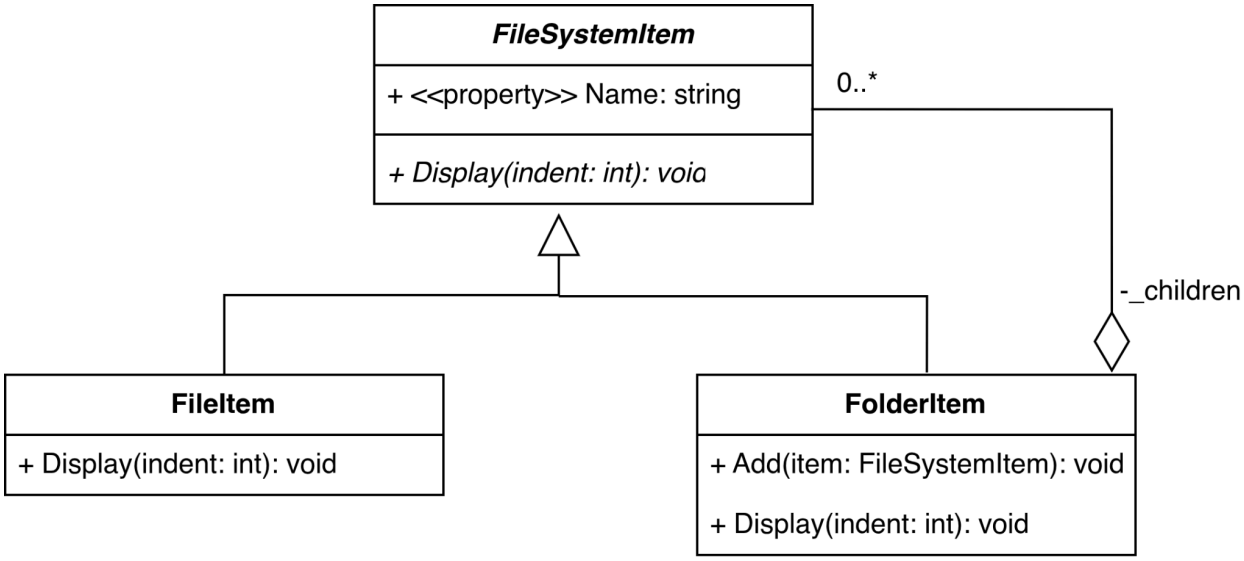
```
{
    public void Execute(IComponent component) => component.Operation();
}

// Entry point
class Program
{
    static void Main()
    {
        // Build tree:
        //   root
        //   ├── leaf1
        //   ├── branch
        //   │   ├── leaf2
        //   │   └── leaf3
        //   └── leaf4

        var root   = new Composite("root");
        var branch = new Composite("branch");
        var leaf1  = new Leaf("leaf1");
        var leaf2  = new Leaf("leaf2");
        var leaf3  = new Leaf("leaf3");
        var leaf4  = new Leaf("leaf4");

        branch.Add(leaf2);
        branch.Add(leaf3);

        root.Add(leaf1);
        root.Add(branch);
        root.Add(leaf4);
```

```
        var client = new Client();

        Console.WriteLine("=== Full tree traversal ===");
        client.Execute(root);

        Console.WriteLine("\n=== Single leaf traversal ===");
        client.Execute(leaf1);

        Console.WriteLine("\n=== Sub-tree traversal ===");
        client.Execute(branch);

        Console.WriteLine("\n=== GetChild demo: root.GetChild(1).Operation()
===");
        root.GetChild(1).Operation();
    }
}
```

Example

File System

ក្នុង Computer យើងមាន File (ឯកសារទោល) និង Folder (ថតឯកសារដែលផ្ទុក File ឬ Folder ផ្សេងទៀត)។ យើងចង់គណនាទំហំ (Size) សរុប ដោយមិនចាំបាច់ខ្វល់ថាវាជា File ឬជា Folder ឡើយ។

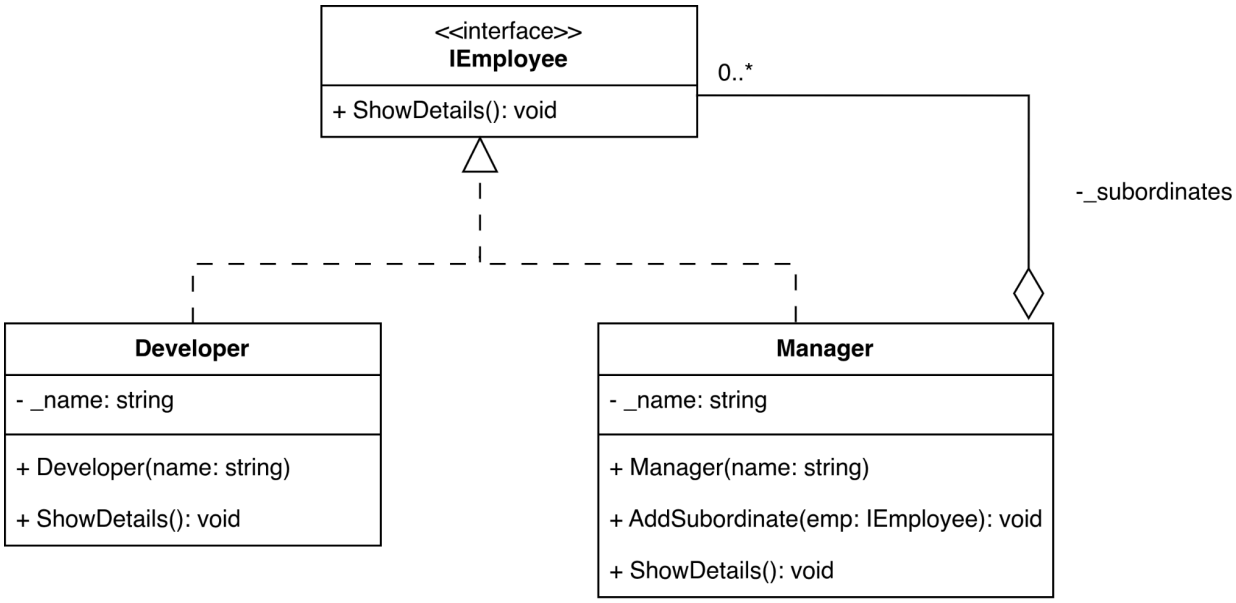



```
using System;
using System.Collections.Generic;

// 1. Component
public abstract class FileSystemItem {
    public string Name { get; set; }
    public abstract void Display(int indent);
}

// 2. Leaf (ឯកសារទោល)
public class FileItem : FileSystemItem {
    public override void Display(int indent) =>
        Console.WriteLine(new string('-', indent) + " File: " + Name);
}
```

```
// 3. Composite (ថតឯកសារ)
public class FolderItem : FileSystemItem {
    private List<FileSystemItem> _children = new List<FileSystemItem>();

    public void Add(FileSystemItem item) => _children.Add(item);

    public override void Display(int indent) {
        Console.WriteLine(new string('-', indent) + " Folder: " + Name);
        foreach (var item in _children) {
            item.Display(indent + 2);
        }
    }
}
```

Organization Structure

ក្នុងក្រុមហ៊ុនមួយ យើងមាន បុគ្គលិកធម្មតា (Employee) និង អ្នកដឹកនាំ (Manager) ដែលគាត់មានកូនចៅក្រោមបង្គាប់។ យើងចង់បង្ហាញឈ្មោះបុគ្គលិកទាំងអស់ក្នុងផ្នែកណាមួយ មិនថាគេជាបុគ្គលិកទោល ឬជាប្រធានផ្នែកដែលមានកូនចៅច្រើននោះទេ។



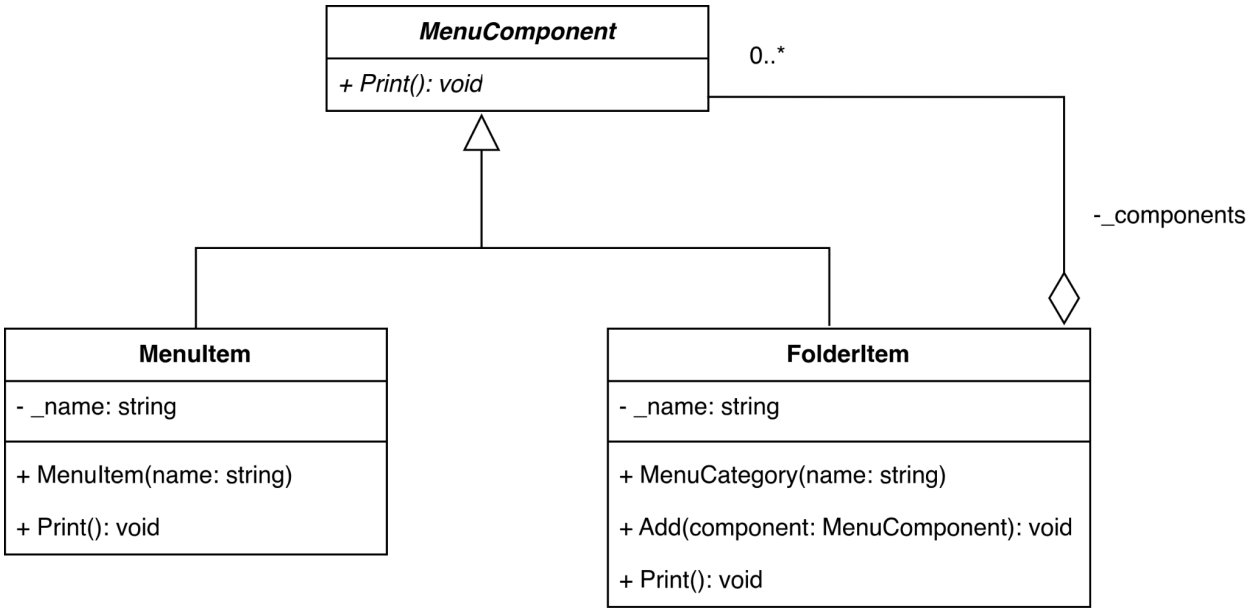

```
// 1. Component
```

```
public interface IEmployee {
    void ShowDetails();
}

// 2. Leaf (បុគ្គលិកធម្មតា)
public class Developer : IEmployee {
    private string _name;
    public Developer(string name) => _name = name;
    public void ShowDetails() => Console.WriteLine($"   Dev: {_name}");
}

// 3. Composite (អ្នកដឹកនាំ/ប្រធានផ្នែក)
public class Manager : IEmployee {
    private string _name;
    private List<IEmployee> _subordinates = new List<IEmployee>();

    public Manager(string name) => _name = name;
    public void AddSubordinate(IEmployee emp) => _subordinates.Add(emp);

    public void ShowDetails() {
        Console.WriteLine($"Manager: {_name}");
        foreach (var emp in _subordinates) {
            emp.ShowDetails();
        }
    }
}
```

Restaurant Menu

ភោជនីយដ្ឋានមាន មុខម្ហូប (Menu Item) និង ប្រភេទម្ហូប (Menu Category ដូចជា៖ ភេសជ្ជ:, បង្អែម)។ ក្នុងប្រភេទភេសជ្ជ: អាចមានប្រភេទរងទៀត (ដូចជា៖ កាហ្វេ, ទឹកផ្លែឈើ)។ យើងចង់ចោះពុម្ព

Menu ទាំងមូលចេញមក។




```
// 1. Component
public abstract class MenuComponent {
    public abstract void Print();
}

// 2. Leaf (មុខម្ហូប)
public class MenuItem : MenuComponent {
    private string _name;
    public MenuItem(string name) => _name = name;
    public override void Print() => Console.WriteLine($" Item: {_name}");
}

// 3. Composite (ប្រភេទម្ហូប)
public class MenuCategory : MenuComponent {
    private string _name;
    private List<MenuComponent> _components = new
List<MenuComponent>();
```

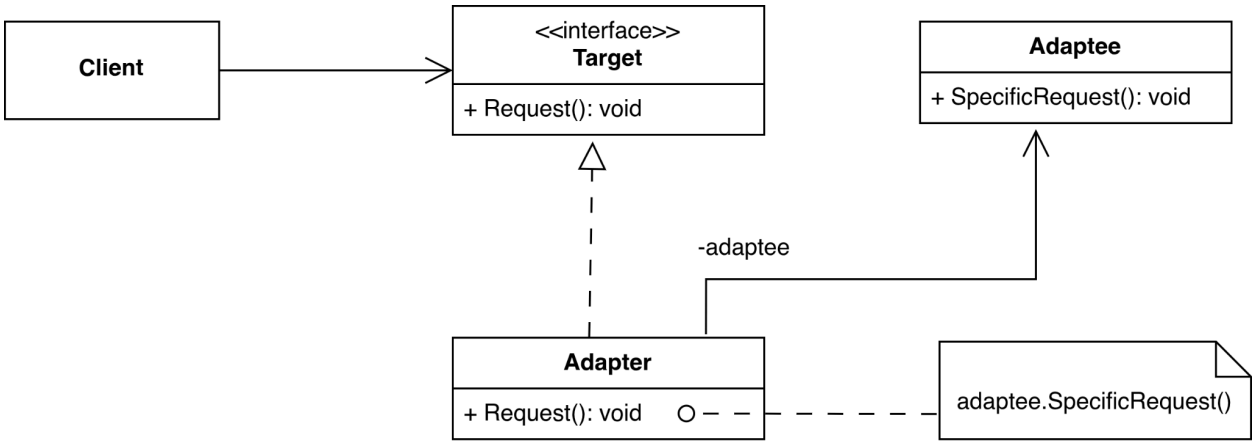
```
    public MenuCategory(string name) => _name = name;
    public void Add(MenuComponent component) =>
_components.Add(component);

    public override void Print() {
        Console.WriteLine($"\n--- Category: {_name} ---");
        foreach (var item in _components) {
            item.Print();
        }
    }
}
```

Adapter Pattern

Intent

បំប្លែង Interface នៃ Class មួយ ទៅជា Interface ផ្សេងទៀតដែល Client រំពឹងទុក។ Adapter អនុញ្ញាតឱ្យ Class ផ្សេងៗគ្នាអាចធ្វើការជាមួយគ្នាបាន ទោះបីជាពួកវាមាន Interface មិនស៊ីគ្នា (Incompatible) ក៏ដោយ។

UML Class Diagram



- Target
  - defines the domain-specific interface that Client uses
- Client
  - collaborates with objects conforming to the Target interface.
- Adaptee
  - defines an existing interface that needs adapting
- Adapter
  - adapts the interface of Adaptee to the Target interface

Sample Code

```
using System;

// Target interface - what the Client expects
```

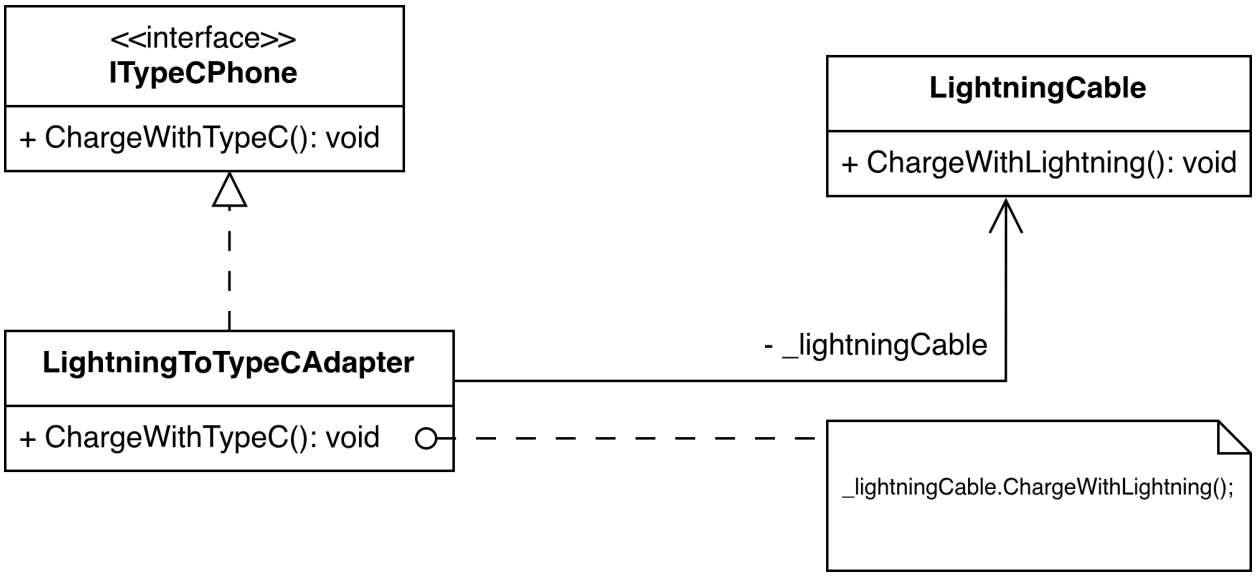
```
public interface ITarget
{
    void Request();
}

// Adaptee - the existing class with an incompatible interface
public class Adaptee
{
    public void SpecificRequest() =>
        Console.WriteLine("Adaptee: SpecificRequest() called.");
}

// Adapter - implements ITarget and wraps the Adaptee
public class Adapter : ITarget
{
    private readonly Adaptee _adaptee;

    public Adapter(Adaptee adaptee) => _adaptee = adaptee;

    // Translates Request() -> adaptee.SpecificRequest()
    public void Request()
    {
        Console.WriteLine("Adapter: Request() called -- delegating to
adaptee.SpecificRequest()");
        _adaptee.SpecificRequest();
    }
}

// Client - works only with ITarget
public class Client
{
```

```
    private readonly ITarget _target;

    public Client(ITarget target) => _target = target;

    public void Execute()
    {
        Console.WriteLine("Client: calling Request() on ITarget...");
        _target.Request();
    }
}

// Entry point
class Program
{
    static void Main()
    {
        // The adaptee has an incompatible interface
        var adaptee = new Adaptee();

        // Wrap it in an adapter that speaks the Target interface
        ITarget adapter = new Adapter(adaptee);

        // Client only knows about ITarget -- completely unaware of Adaptee
        var client = new Client(adapter);
        client.Execute();
    }
}
```

Example

Lightning to Type-C

អ្នកមានខ្សែសាក iPhone (Lightning) ប៉ុន្តែទូរស័ព្ទថ្មីរបស់អ្នកប្រើរន្ធ USB Type-C។ អ្នកមិនចង់ចោះខ្សែចាស់ចោលទេ ដូច្នេះអ្នកត្រូវការ Adapter ដើម្បីប្រើខ្សែ Lightning ជាមួយរន្ធ Type-C។

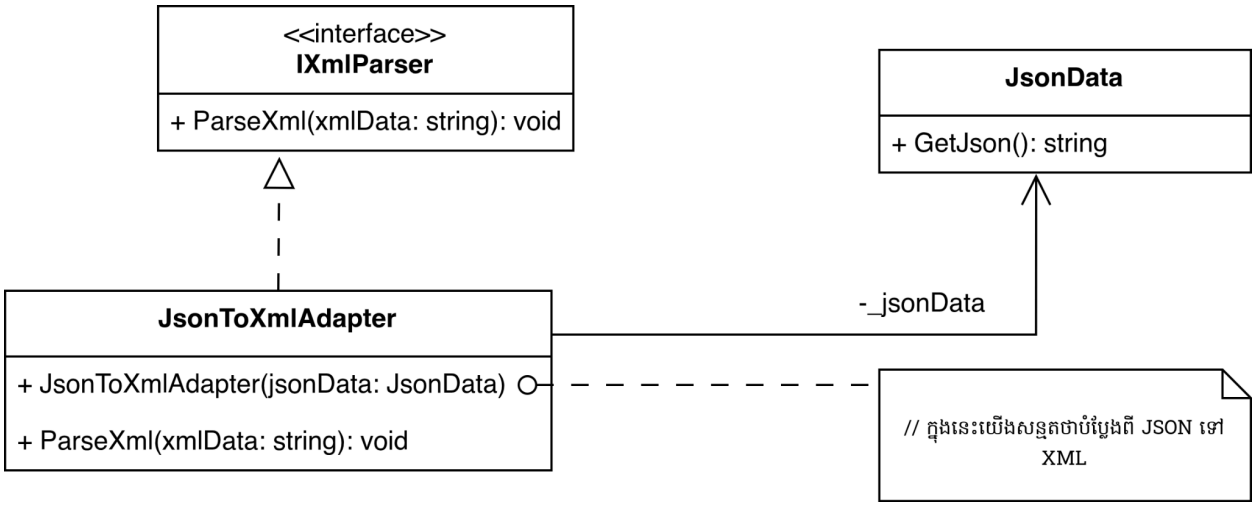



```
using System;

// 1. Target Interface (អ្វីដែលទូរស័ព្ទចង់បាន)
public interface ITypeCPhone {
    void ChargeWithTypeC();
}

// 2. Adaptee (របស់ដែលយើងមានស្រាប់ តែមិនត្រូវរន្ធ)
public class LightningCable {
    public void ChargeWithLightning() => Console.WriteLine("Charging with
Lightning cable...");
}

// 3. Adapter (អ្នកបំប្លែង)
```

```
public class LightningToTypeCAdapter : ITypeCPhone {
    private readonly LightningCable _lightningCable;

    public LightningToTypeCAdapter(LightningCable lightningCable) {
        _lightningCable = lightningCable;
    }

    public void ChargeWithTypeC() {
        Console.WriteLine("Adapter converting Type-C to Lightning...");
        _lightningCable.ChargeWithLightning();
    }
}
```

JSON to XML

កម្មវិធីរបស់អ្នកប្រើប្រាស់ទិន្នន័យជា JSON ប៉ុន្តែអ្នកចង់ប្រើ Library ក្រៅមួយ (Third-party) ដែលតម្រូវឱ្យបញ្ចូលទិន្នន័យជា XML ទើបវាដំណើរការ។

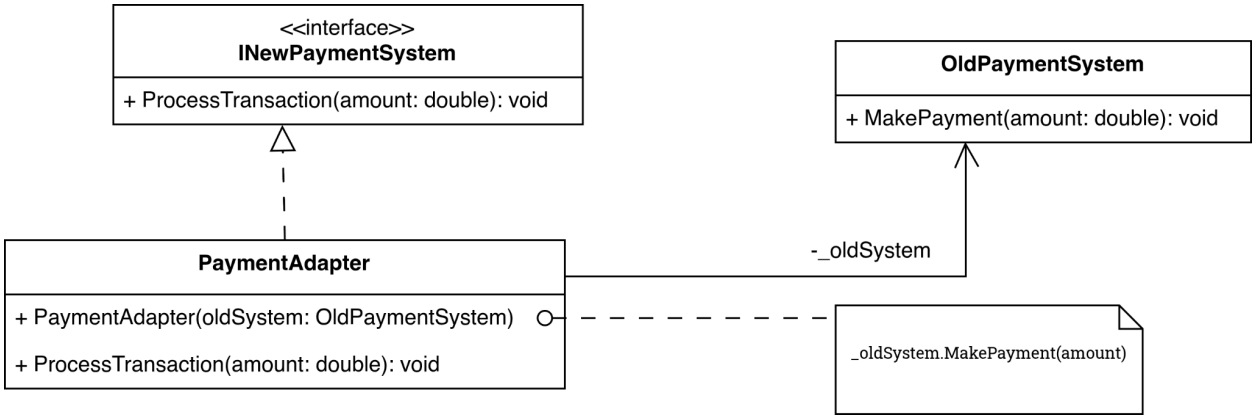



```
// 1. Target (អ្វីដែលកម្មវិធីយើងរំពឹងទុក)
public interface IXmlParser {
    void ParseXml(string xmlData);
}
```

```
// 2. Adaptee (របស់ដែលមានស្រាប់ - JSON)
public class JsonData {
    public string GetJson() => "{ 'name': 'Dara', 'age': 25 }";
}

// 3. Adapter (អ្នកបំប្លែង JSON ទៅ XML)
public class JsonToXmlAdapter : IXmlParser {
    private readonly JsonData _jsonData;

    public JsonToXmlAdapter(JsonData jsonData) => _jsonData = jsonData;

    public void ParseXml(string xmlData) {
        // ក្នុងនេះយើងសន្មតថាបំប្លែងពី JSON ទៅ XML
        string convertedData = "<user><name>Dara</name></user>";
        Console.WriteLine($"Parsing converted XML: {convertedData}");
    }
}
```

Legacy Payment System

ក្រុមហ៊ុនរបស់អ្នកមានប្រព័ន្ធទូទាត់ប្រាក់ចាស់ (Legacy System) ដែលប្រើ Method ឈ្មោះ MakePayment()។ ប៉ុន្តែប្រព័ន្ធថ្មីដែលអ្នកទើបនឹងទិញមក ប្រើ Method ឈ្មោះ ProcessTransaction()។ ដើម្បីកុំឱ្យកែរកូដចាស់ច្រើន យើងប្រើ Adapter។



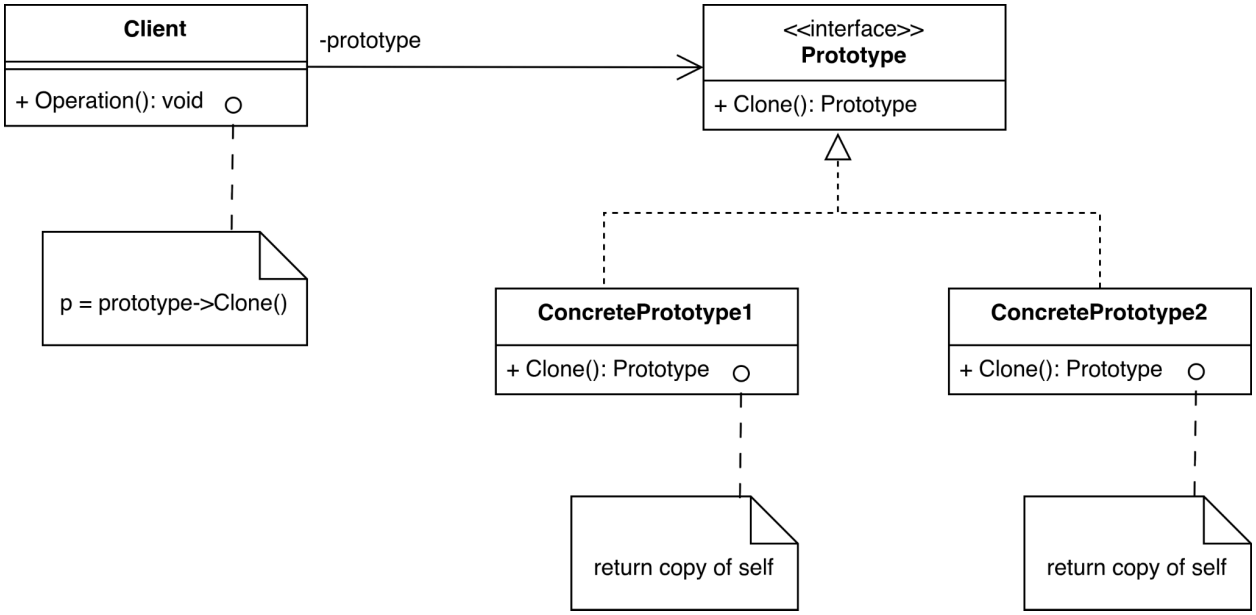
```
// 1. Target
public interface INewPaymentSystem {
    void ProcessTransaction(double amount);
}

// 2. Adaptee (ប្រព័ន្ធចាស់)
public class OldPaymentSystem {
    public void MakePayment(double amount) {
        Console.WriteLine($"Payment of ${amount} processed via Legacy
System.");
    }
}

// 3. Adapter
public class PaymentAdapter : INewPaymentSystem {
    private readonly OldPaymentSystem _oldSystem;

    public PaymentAdapter(OldPaymentSystem oldSystem) => _oldSystem =
oldSystem;

    public void ProcessTransaction(double amount) {
        // បញ្ជូនបន្តទៅកាន់ Method របស់ប្រព័ន្ធចាស់
        _oldSystem.MakePayment(amount);
    }
}
```

Prototype Pattern

Intent

កំណត់ប្រភេទនៃវត្ថុ (Objects) ដែលត្រូវបង្កើតដោយប្រើគំរូដើម (Prototypical instance) ហើយបង្កើតវត្ថុថ្មីៗតាមរយៈការចម្លង (Copying) ចេញពីគំរូដើមនោះ។

UML Class Diagram



- Prototype
  - declares an interface for cloning itself
- ConcretePrototype
  - implements an operation for cloning itself
- Client
  - creates a new object by asking a prototype to clone itself

Sample Code

```
using System;

// Prototype interface
```

```
public interface IPrototype
{
    IPrototype Clone();
}

// ConcretePrototype1
public class ConcretePrototype1 : IPrototype
{
    public string Name { get; set; }
    public int Value { get; set; }

    public ConcretePrototype1(string name, int value)
    {
        Name  = name;
        Value = value;
    }

    // return copy of self
    public IPrototype Clone()
    {
        Console.WriteLine($"ConcretePrototype1: cloning [{Name}, {Value}]");
        return new ConcretePrototype1(Name, Value);
    }

    public override string ToString() =>
        $"ConcretePrototype1 {{ Name = {Name}, Value = {Value} }}";
}

// ConcretePrototype2
public class ConcretePrototype2 : IPrototype
{
```

```
    public string Label     { get; set; }
    public double Factor    { get; set; }

    public ConcretePrototype2(string label, double factor)
    {
        Label  = label;
        Factor = factor;
    }

    // return copy of self
    public IPrototype Clone()
    {
        Console.WriteLine($"ConcretePrototype2: cloning [{Label}, {Factor}]");
        return new ConcretePrototype2(Label, Factor);
    }

    public override string ToString() =>
        $"ConcretePrototype2 {{ Label = {Label}, Factor = {Factor} }}";
}

// Client
public class Client
{
    private IPrototype _prototype;   // -prototype association

    public Client(IPrototype prototype) => _prototype = prototype;

    // Operation note: p = prototype->Clone()
    public void Operation()
    {
        Console.WriteLine("Client: calling Operation() -- cloning prototype...");
```

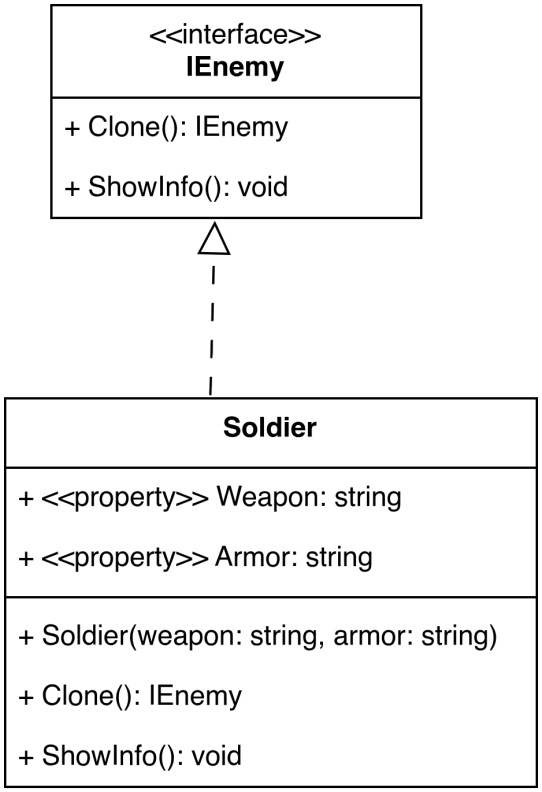
```
        IPrototype p = _prototype.Clone();
        Console.WriteLine($"Client: clone produced → {p}");
    }
}

// Entry point
class Program
{
    static void Main()
    {
        // --- ConcretePrototype1 ---
        var proto1  = new ConcretePrototype1("Alpha", 42);
        var client1 = new Client(proto1);
        client1.Operation();

        Console.WriteLine();

        // --- ConcretePrototype2 ---
        var proto2  = new ConcretePrototype2("Beta", 3.14);
        var client2 = new Client(proto2);
        client2.Operation();

        Console.WriteLine();

        // --- Demonstrate independence of clone from original ---
        var original = new ConcretePrototype1("Original", 100);
        var clone    = (ConcretePrototype1)original.Clone();
        clone.Name   = "Clone";
        clone.Value  = 999;

        Console.WriteLine($"Original after mutating clone → {original}");
```

```
        Console.WriteLine($"Clone                    → {clone}");
    }
}
```

Example

Game Character Settings

ក្នុងហ្គេមបាញ់គ្នា អ្នកមានទាហាន (Soldier) រាប់រយនាក់ដែលមានរូបរាងដូចគ្នា។ បើអ្នកបង្កើតទាហានម្នាក់ៗដោយទាញយកម៉ូដែល 3D និង Texture ពីមេម៉ូរីរាល់ដង វានឹងធ្វើឱ្យហ្គេមដើរយឺត។

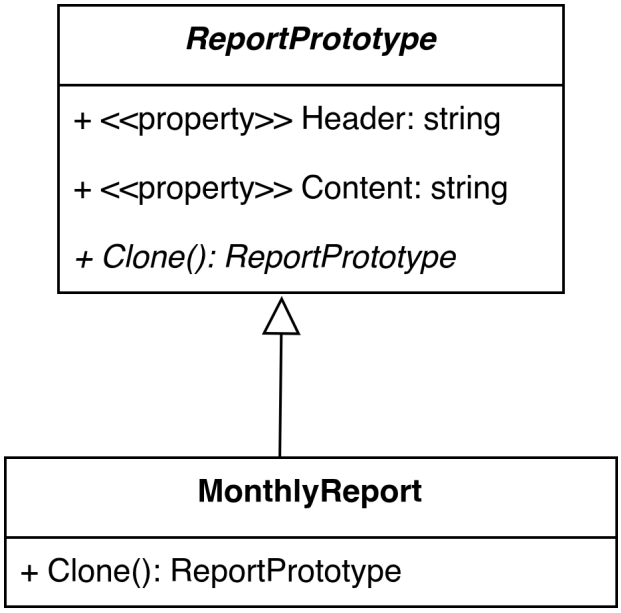



```
using System;

// 1. Prototype Interface
public interface IEnemy {
    IEnemy Clone();
```

```
    void ShowInfo();
}

// 2. Concrete Prototype
public class Soldier : IEnemy {
    public string Weapon { get; set; }
    public string Armor { get; set; }

    public Soldier(string weapon, string armor) {
        this.Weapon = weapon;
        this.Armor = armor;
        // សន្មតថាការបង្កើតដំបូងនេះស៊ីពេលខ្លាំង (Loading 3D Model)
    }

    public IEnemy Clone() {
        // បង្កើត Object ថ្មីដោយចម្លងតម្លៃពី Object បច្ចុប្បន្ន (Shallow Copy)
        return (IEnemy)this.MemberwiseClone();
    }

    public void ShowInfo() => Console.WriteLine($"Soldier with {Weapon} and
{Armor} armor.");
}
```

Standard Document Templates

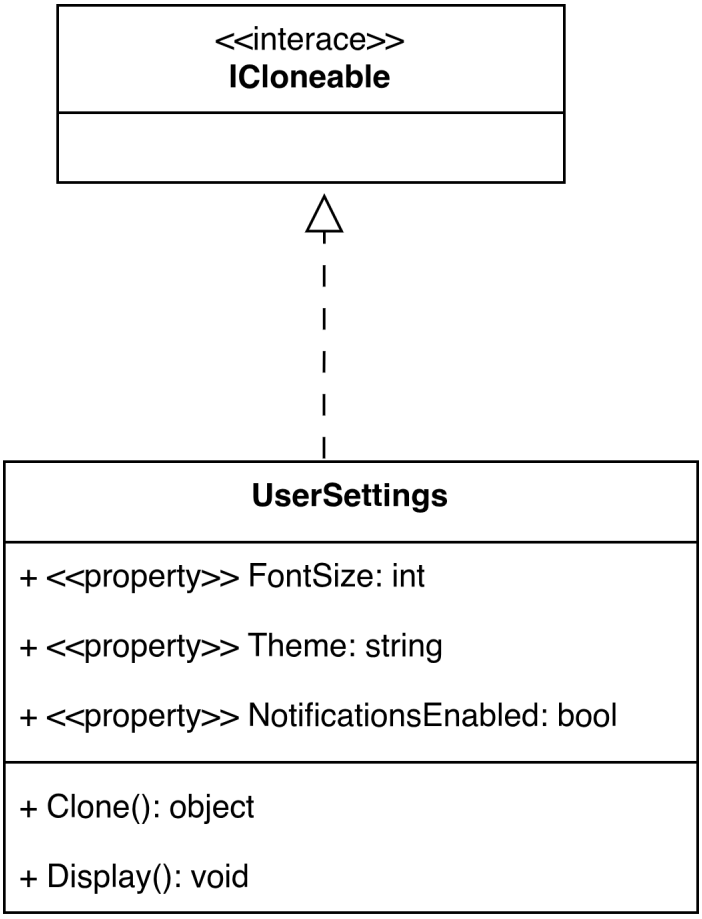
ក្រុមហ៊ុនមួយមានទម្រង់របាយការណ៍ (Report Template) ដែលមាន Header, Footer និង Logo ដូចៗគ្នា។ រាល់ពេលបង្កើតរបាយការណ៍ថ្មី យើងមិនចង់កំណត់ Header/Footer ម្តងទៀតទេ យើងគ្រាន់តែចង់ចម្លងទម្រង់ដើម រួចប្តូរតែខ្លឹមសារខាងក្នុងបានហើយ។




```
// 1. Prototype
public abstract class ReportPrototype {
    public string Header { get; set; }
    public string Content { get; set; }
    public abstract ReportPrototype Clone();
}

// 2. Concrete Prototype
public class MonthlyReport : ReportPrototype {
    public override ReportPrototype Clone() {
        Console.WriteLine("Cloning monthly report template...");
        return (ReportPrototype)this.MemberwiseClone();
    }
}
```

User Settings/Profile

អ្នកប្រើប្រាស់ចង់បង្កើត Profile ច្រើន (ដូចជា Profile សម្រាប់ធ្វើការ និង Profile សម្រាប់លេងហ្គេម) ប៉ុន្តែពួកគេចង់ឱ្យ Profile ថ្មីមានការកំណត់ (Settings) ភាគច្រើនដូចទៅនឹង Profile ដើមដែរ ដើម្បីកុំឱ្យពិបាកកំណត់ម្តងម្នាក់ៗពីសូន្យ។




```
// 1. Concrete Prototype
public class UserSettings : ICloneable {
    public int FontSize { get; set; }
    public string Theme { get; set; }
    public bool NotificationsEnabled { get; set; }

    public object Clone() {
        // ប្រើ Interface ដែលមានស្រាប់ក្នុង .NET គឺ ICloneable
        return this.MemberwiseClone();
    }

    public void Display() =>
        Console.WriteLine($"Theme: {Theme}, Font: {FontSize}, Notif:
{NotificationsEnabled}");
```

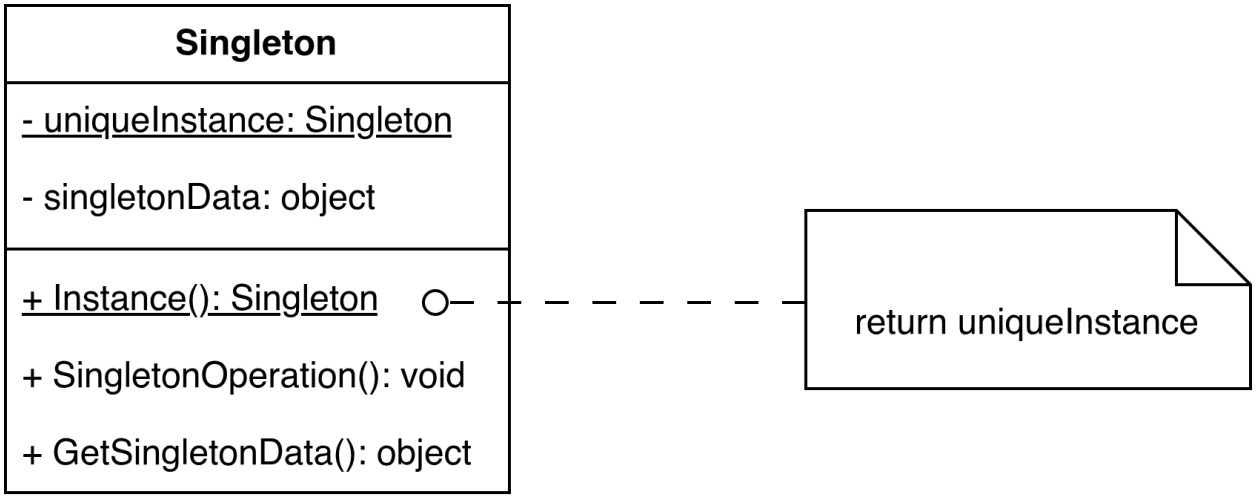
```
}

// ការប្រើប្រាស់៖
// UserSettings defaultSettings = new UserSettings { Theme = "Dark", FontSize
= 14, NotificationsEnabled = true };
// UserSettings workSettings = (UserSettings)defaultSettings.Clone();
// workSettings.FontSize = 12; // កែតែចំណុចដែលខុសគ្នា
```

Singleton Pattern

Intent

ធានាថា Class មួយមាន Instance (វត្ថុដែលបង្កើតចេញពី Class) តែមួយគត់
និងផ្តល់នូវចំណុចចូលប្រើប្រាស់ជាសកល (Global point of access) ទៅកាន់ Instance នោះ។

UML Class Diagram



- Singleton
  - defines an Instance operation that lets clients access its unique instance. Instance is a class operation
  - may be responsible for creating its own unique instance

Sample Code

```
using System;
using System.Threading;

public sealed class Singleton
{
    // - uniqueInstance: Singleton  (static = underlined in UML)
    private static Singleton _uniqueInstance;
```

```
private static readonly Lock _lock = new();

// - singletonData: object
private object _singletonData;

// Private constructor -- prevents external instantiation
private Singleton()
{
    _singletonData = "Initial singleton data";
    Console.WriteLine("Singleton: instance created.");
}

// + Instance(): Singleton  (static = underlined in UML)
// note: return uniqueInstance
public static Singleton Instance()
{
    if (_uniqueInstance == null)
    {
        lock (_lock)
        {
            if (_uniqueInstance == null)          // double-checked locking
                _uniqueInstance = new Singleton();
        }
    }
    return _uniqueInstance;
}

// + SingletonOperation(): void
public void SingletonOperation()
{
    Console.WriteLine("Singleton: SingletonOperation() called.");
```

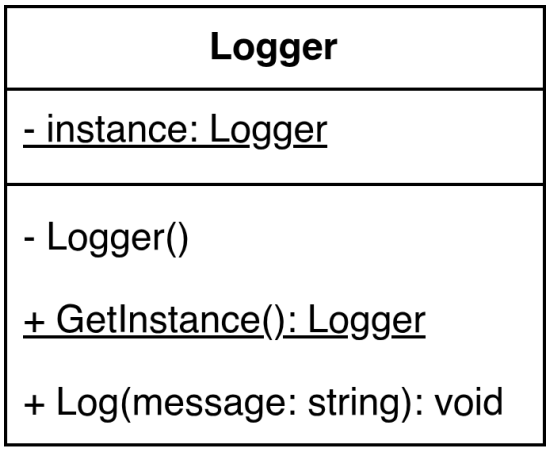
```
    }

    // + GetSingletonData(): object
    public object GetSingletonData() => _singletonData;

    // Optional setter to demonstrate shared state
    public void SetSingletonData(object data)
    {
        _singletonData = data;
        Console.WriteLine($"Singleton: singletonData updated to
[{_singletonData}]");
    }
}

// Entry point
class Program
{
    static void Main()
    {
        Console.WriteLine("=== Acquiring instance (first call -- triggers
construction) ===");
        var s1 = Singleton.Instance();
        s1.SingletonOperation();
        Console.WriteLine($"s1 data: {s1.GetSingletonData()}");

        Console.WriteLine();
        Console.WriteLine("=== Acquiring instance (second call -- reuses
existing) ===");
        var s2 = Singleton.Instance();
        s2.SingletonOperation();
        Console.WriteLine($"s2 data: {s2.GetSingletonData()}");
```

```
        Console.WriteLine();
        Console.WriteLine("=== Proving s1 and s2 are the same instance ===");
        Console.WriteLine($"ReferenceEquals(s1, s2): {ReferenceEquals(s1, s2)}");

        Console.WriteLine();
        Console.WriteLine("=== Mutating shared state via s2, visible through s1
===");
        s2.SetSingletonData("Updated shared data");
        Console.WriteLine($"s1 data after s2 mutation: {s1.GetSingletonData()}");

        Console.WriteLine();
        Console.WriteLine("=== Thread-safety demo: 5 threads all get same
instance ===");
        for (int i = 1; i <= 5; i++)
        {
            int threadId = i;
            new Thread(() =>
            {
                var s = Singleton.Instance();
                Console.WriteLine($"Thread {threadId}: hash = {s.GetHashCode()}");
            }).Start();
        }
    }
}
```

Example

Log Manager

ក្នុងកម្មវិធីដ៏ធំមួយ មានផ្នែកជាច្រើនត្រូវការសរសេរ Log (កំណត់ហេតុ) ទៅក្នុង File តែមួយ។
ប្រសិនបើគ្រប់ផ្នែកទាំងអស់បង្កើត Object សម្រាប់សរសេរ File រៀងៗខ្លួន
វានឹងធ្វើឱ្យមានបញ្ហាជាន់គ្នាក្នុងការបើក File (File Access Conflict) និងស៊ីមេម៉ូរីឥតប្រយោជន៍។




```
using System;

public class Logger {
    // ១. បង្កើត Static Instance តែមួយគត់
    private static Logger _instance;

    // ២. បង្កើត Private Constructor ដើម្បីការពារកុំឱ្យគេប្រើ 'new' ពីខាងក្រៅ
    private Logger() { }

    // ៣. បង្កើត Static Method សម្រាប់ឱ្យគេហៅប្រើ
    public static Logger GetInstance() {
        if (_instance == null) {
            _instance = new Logger();
        }
        return _instance;
    }
```

```
    public void Log(string message) {
        Console.WriteLine($"[LOG]: {message}");
    }
}
```

App Settings

កម្មវិធីរបស់អ្នកមានការកំណត់រួម (Settings) ដូចជា ភាសា (Language) ឬ ពណ៌ (Theme)។ អ្នកចង់ឱ្យរាល់គ្រប់ Page ឬគ្រប់ Component ទាំងអស់ប្រើប្រាស់ទិន្នន័យ Settings តែមួយរួមគ្នា ដើម្បីឱ្យនៅពេលប្តូរភាសានៅកន្លែងមួយ កន្លែងផ្សេងទៀតក៏ប្តូរតាមដែរ។

AppSettings
- instance: AppSettings + <<property>> Language: string = "Khmer"
- AppSettings() + GetInstance(): AppSettings

```
public class AppSettings {
    private static AppSettings _instance;
    public string Language { get; set; } = "Khmer";

    private AppSettings() { }

    public static AppSettings GetInstance() {
        if (_instance == null) {
            _instance = new AppSettings();
        }
        return _instance;
    }
```

```
}

// ការប្រើប្រាស់៖
// var settings = AppSettings.GetInstance();
// settings.Language = "English";
```

Database Connection Pool

ការបង្កើតការភ្ជាប់ (Connection) ទៅកាន់ Database ម្តងៗស៊ីពេល និងកម្លាំងម៉ាស៊ីនខ្លាំងៗ យើងមិនចង់ឱ្យរាល់ Query ម្តងៗបង្កើត Connection ថ្មីនោះទេ យើងចង់ឱ្យមាន Object មួយដែលគ្រប់គ្រង Connection ទាំងនោះសម្រាប់ប្រើរួមគ្នាពេញមួយកម្មវិធី។

DatabaseConnection
- instance: DatabaseConnection
- DatabaseConnection() + GetInstance(): DatabaseConnection + ExecuteQuery(sql: string): void

```
public class DatabaseConnection {
    private static DatabaseConnection _instance;

    private DatabaseConnection() {
        // កូដសម្រាប់ភ្ជាប់ទៅ Database
        Console.WriteLine("Database Connected.");
    }

    public static DatabaseConnection GetInstance() {
        if (_instance == null) {
            _instance = new DatabaseConnection();
        }
```

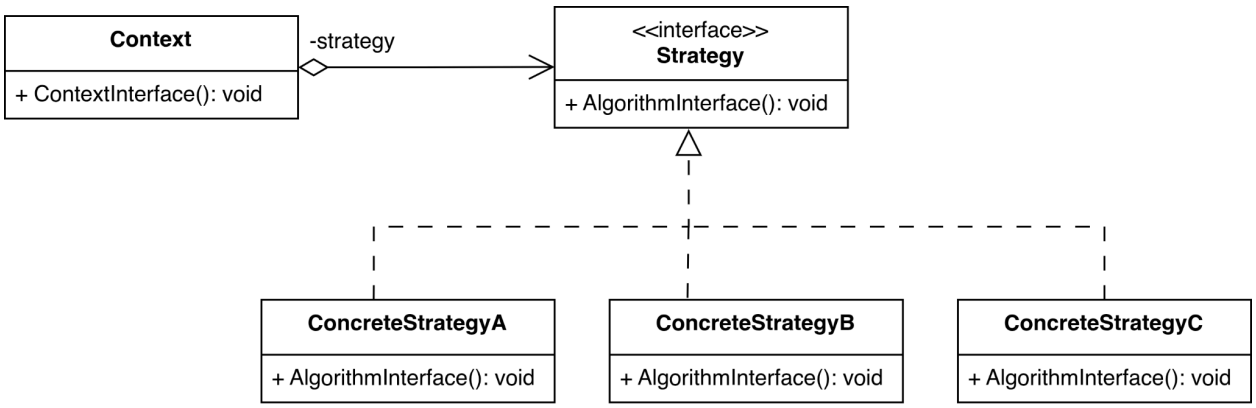
```
        return _instance;
    }

    public void ExecuteQuery(string sql) {
        Console.WriteLine($"Executing: {sql}");
    }
}
```

Strategy Pattern

Intent

កំណត់ក្រុមនៃក្បួនដោះស្រាយ (Family of algorithms) ដោយវេចខ្ជប់ពួកវាឱ្យដាច់ដោយឡែកពីគ្នា (Encapsulate) និងធ្វើឱ្យពួកវាអាចផ្លាស់ប្តូរគ្នាទៅវិញទៅមកបាន (Interchangeable)។ Strategy អនុញ្ញាតឱ្យក្បួនដោះស្រាយផ្លាស់ប្តូរដោយឯករាជ្យ ពី Client ដែលប្រើប្រាស់វា។

UML Class Diagram



- Strategy
  - declares an interface common to all supported algorithms. Context uses this interface to call the algorithm defined by a ConcreteStrategy.
- ConcreteStrategy
  - implements the algorithm using the Strategy interface.
- Context
  - is configured with a ConcreteStrategy object.
  - maintains a reference to a Strategy object.
  - may define an interface that lets Strategy access its data.

Sample Code

```
using System;

// Strategy interface
```

```
public interface IStrategy
{
    void AlgorithmInterface();
}

// ConcreteStrategyA
public class ConcreteStrategyA : IStrategy
{
    public void AlgorithmInterface() =>
        Console.WriteLine("ConcreteStrategyA: executing algorithm A.");
}

// ConcreteStrategyB
public class ConcreteStrategyB : IStrategy
{
    public void AlgorithmInterface() =>
        Console.WriteLine("ConcreteStrategyB: executing algorithm B.");
}

// ConcreteStrategyC
public class ConcreteStrategyC : IStrategy
{
    public void AlgorithmInterface() =>
        Console.WriteLine("ConcreteStrategyC: executing algorithm C.");
}

// Context - holds a reference to a Strategy (aggregation: -strategy)
public class Context
{
    private IStrategy _strategy;   // -strategy aggregation
```

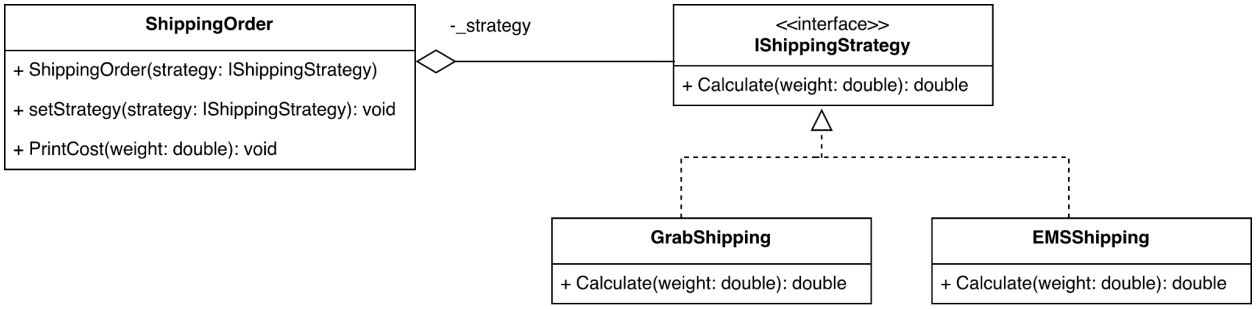
```
    public Context(IStrategy strategy) => _strategy = strategy;

    // Allow swapping strategy at runtime
    public void SetStrategy(IStrategy strategy) => _strategy = strategy;

    // Delegates work to whichever strategy is currently set
    public void ContextInterface()
    {
        Console.WriteLine("Context: calling ContextInterface() -- delegating to
strategy...");
        _strategy.AlgorithmInterface();
    }
}

// Entry point
class Program
{
    static void Main()
    {
        // Start with Strategy A
        var context = new Context(new ConcreteStrategyA());
        context.ContextInterface();

        Console.WriteLine();

        // Swap to Strategy B at runtime
        context.SetStrategy(new ConcreteStrategyB());
        context.ContextInterface();

        Console.WriteLine();
```

```
        // Swap to Strategy C at runtime
        context.SetStrategy(new ConcreteStrategyC());
        context.ContextInterface();
    }
}
```

Example

Shipping Cost

ហាងលក់ទំនិញអនឡាញមានជម្រើសដឹកជញ្ជូនច្រើនដូចជា៖ ដឹកតាម Grab, ដឹកតាមក្រុមហ៊ុនដឹកជញ្ជូន (EMS), ឬដឹកដោយខ្លួនឯង។ ការគណនាតម្លៃសម្រាប់សេវាកម្មនីមួយៗគឺខុសគ្នាទាំងស្រុង។ បើយើងប្រើ if-else ច្រើន វានឹងធ្វើឱ្យកូដពិបាកកែប្រែនៅពេលមានក្រុមហ៊ុនថ្មីបន្ថែម។

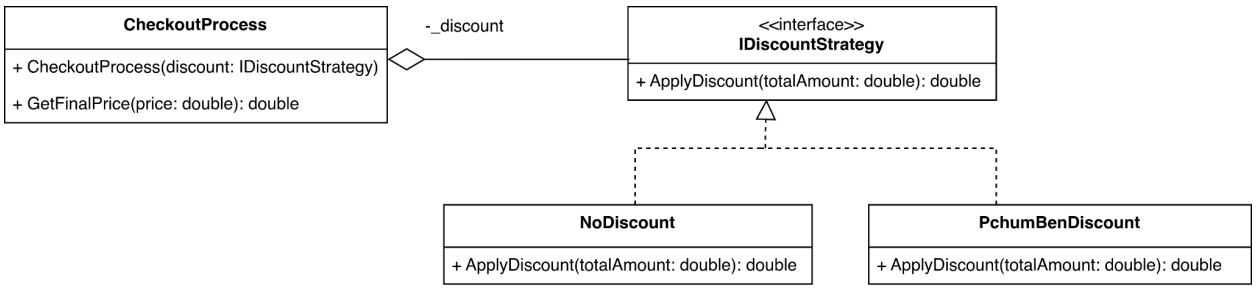



```
using System;

// 1. Strategy Interface
public interface IShippingStrategy {
    double Calculate(double weight);
}

// 2. Concrete Strategies
public class GrabShipping : IShippingStrategy {
    public double Calculate(double weight) => weight * 1.5; // $1.5 ក្នុងមួយគីឡូ
}
```

```
public class EMSShipping : IShippingStrategy {
    public double Calculate(double weight) => weight * 2.5; // $2.5 ក្នុងមួយគីឡូ
}

// 3. Context (អ្នកប្រើប្រាស់យុទ្ធសាស្ត្រ)
public class ShippingOrder {
    private IShippingStrategy _strategy;
    public ShippingOrder(IShippingStrategy strategy) => _strategy = strategy;

    public void SetStrategy(IShippingStrategy strategy) => _strategy = strategy;

    public void PrintCost(double weight) {
        Console.WriteLine($"Shipping cost: ${_strategy.Calculate(weight)}");
    }
}
```

Discount System

ក្នុងថ្ងៃបុណ្យភ្ជុំបិណ្ឌ អ្នកចង់ចុះតម្លៃ ១០%, ថ្ងៃបុណ្យចូលឆ្នាំចុះតម្លៃ ២០%, ហើយថ្ងៃធម្មតាមិនចុះតម្លៃទេ។ Strategy Pattern អនុញ្ញាតឱ្យអ្នកប្តូររូបមន្តបញ្ចុះតម្លៃបានយ៉ាងងាយស្រួលតាមថ្ងៃនីមួយៗ។

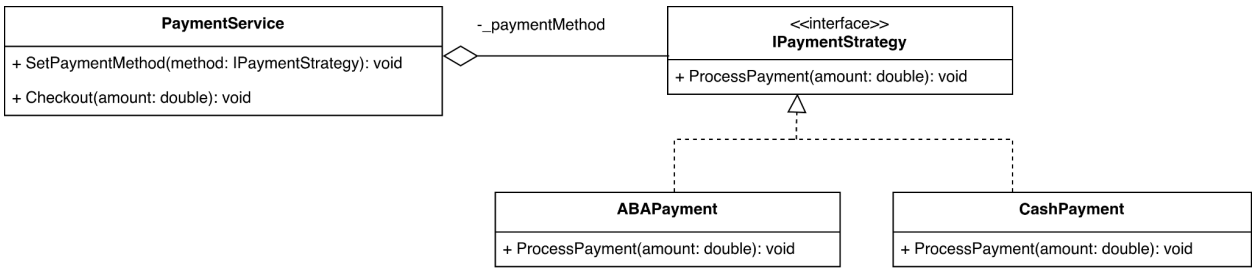



```
// 1. Strategy
public interface IDiscountStrategy {
    double ApplyDiscount(double totalAmount);
}

// 2. Concrete Strategies
```

```
public class NoDiscount : IDiscountStrategy {
    public double ApplyDiscount(double totalAmount) => totalAmount;
}

public class PchumBenDiscount : IDiscountStrategy {
    public double ApplyDiscount(double totalAmount) => totalAmount * 0.9; //
ចុះ 10%
}

// 3. Context
public class CheckoutProcess {
    private IDiscountStrategy _discount;
    public CheckoutProcess(IDiscountStrategy discount) => _discount =
discount;

    public double GetFinalPrice(double price) => _discount.ApplyDiscount(price);
}
```

Payment Methods

អតិថិជនចង់ទូទាត់ប្រាក់តាមរយៈ ABA, Wing, ឬសាច់ប្រាក់សុទ្ធ (Cash)។
រាល់វិធីសាស្ត្រនីមួយៗមានដំណើរការ (Logic) ផ្សេងៗគ្នា។ យើងប្រើ Strategy
ដើម្បីគ្រប់គ្រងរាល់វិធីទូទាត់ទាំងនោះ។




```
// 1. Strategy
public interface IPaymentStrategy {
    void ProcessPayment(double amount);
```

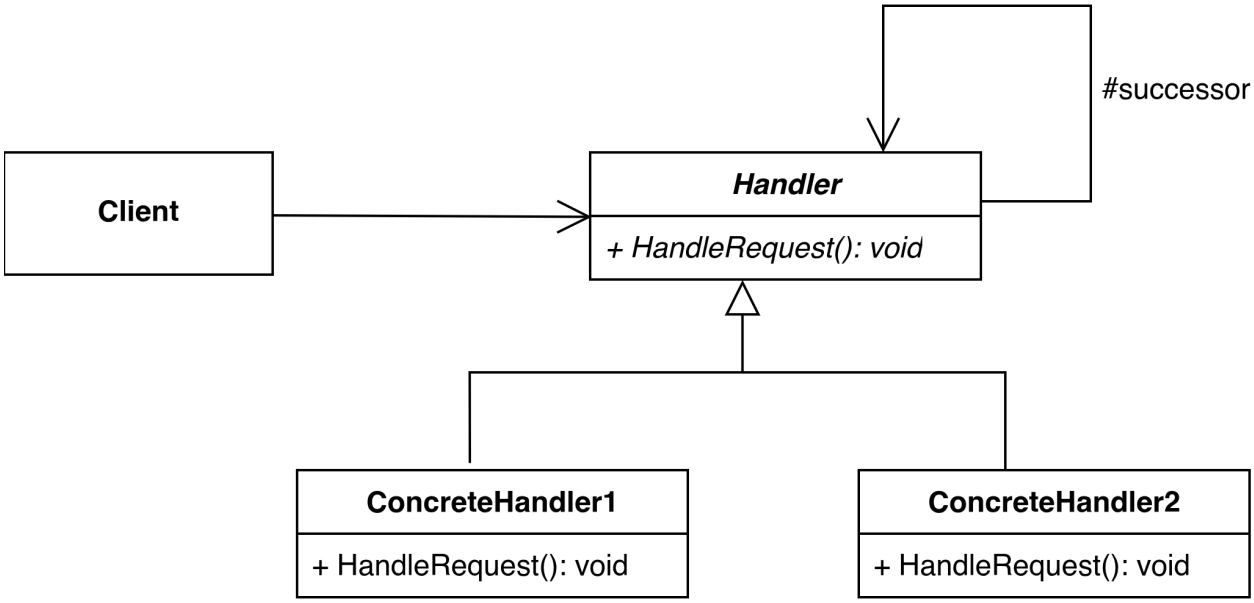
```
}

// 2. Concrete Strategies
public class ABAPayment : IPaymentStrategy {
    public void ProcessPayment(double amount) => Console.WriteLine($"Paid
${amount} via ABA QR Code.");
}

public class CashPayment : IPaymentStrategy {
    public void ProcessPayment(double amount) => Console.WriteLine($"Paid
${amount} with physical cash.");
}

// 3. Context
public class PaymentService {
    private IPaymentStrategy _paymentMethod;
    public void SetPaymentMethod(IPaymentStrategy method) =>
_paymentMethod = method;

    public void Checkout(double amount) {
        _paymentMethod.ProcessPayment(amount);
    }
}
```

Chain of Responsibility Pattern

Intent

ជៀសវាងការចងភ្ជាប់គ្នា (Coupling) រវាងអ្នកផ្ញើសេចក្តីថ្លែងការណ៍ (Sender) និងអ្នកទទួល (Receiver) ដោយអនុញ្ញាតឱ្យមានវត្ថុ (Objects) ច្រើនជាងមួយមានឱកាសដោះស្រាយសេចក្តីថ្លែងការណ៍នោះ។
បង្កើតជាខ្សែសង្វាក់នៃវត្ថុអ្នកទទួល (Chain)
ហើយបញ្ជូនសេចក្តីថ្លែងការណ៍បន្តបន្ទាប់គ្នានៅក្នុងខ្សែសង្វាក់នោះ
រហូតទាល់តែមានវត្ថុណាមួយដោះស្រាយវាបាន។

UML Class Diagram



- Handler
  - defines an interface for handling requests.
  - (optional) implements the successor link.
- ConcreteHandler
  - handles requests it is responsible for.
  - can access its successor.
  - if the ConcreteHandler can handle the request, it does so; otherwise it forwards the request to its successor.
- Client

- initiates the request to a ConcreteHandler object on the chain.

Sample Code

```
using System;

// Abstract Handler - holds #successor self-reference
public abstract class Handler
{
    protected Handler Successor { get; private set; }   // #successor

    public void SetSuccessor(Handler successor) => Successor = successor;

    public abstract void HandleRequest(int request);
}

// ConcreteHandler1 - handles requests in range [0, 10)
public class ConcreteHandler1 : Handler
{
    private const int Min = 0;
    private const int Max = 10;

    public override void HandleRequest(int request)
    {
        if (request >= Min && request < Max)
        {
            Console.WriteLine($"ConcreteHandler1: handled request [{request}]");
        }
        else if (Successor != null)
        {
            Console.WriteLine($"ConcreteHandler1: cannot handle [{request}],
passing to successor...");
```

```
            Successor.HandleRequest(request);
        }
        else
        {
            Console.WriteLine($"ConcreteHandler1: no successor available, request
[{request}] unhandled.");
        }
    }
}

// ConcreteHandler2 - handles requests in range [10, 20)
public class ConcreteHandler2 : Handler
{
    private const int Min = 10;
    private const int Max = 20;

    public override void HandleRequest(int request)
    {
        if (request >= Min && request < Max)
        {
            Console.WriteLine($"ConcreteHandler2: handled request [{request}]");
        }
        else if (Successor != null)
        {
            Console.WriteLine($"ConcreteHandler2: cannot handle [{request}],
passing to successor...");
            Successor.HandleRequest(request);
        }
        else
        {
            Console.WriteLine($"ConcreteHandler2: no successor available, request
```

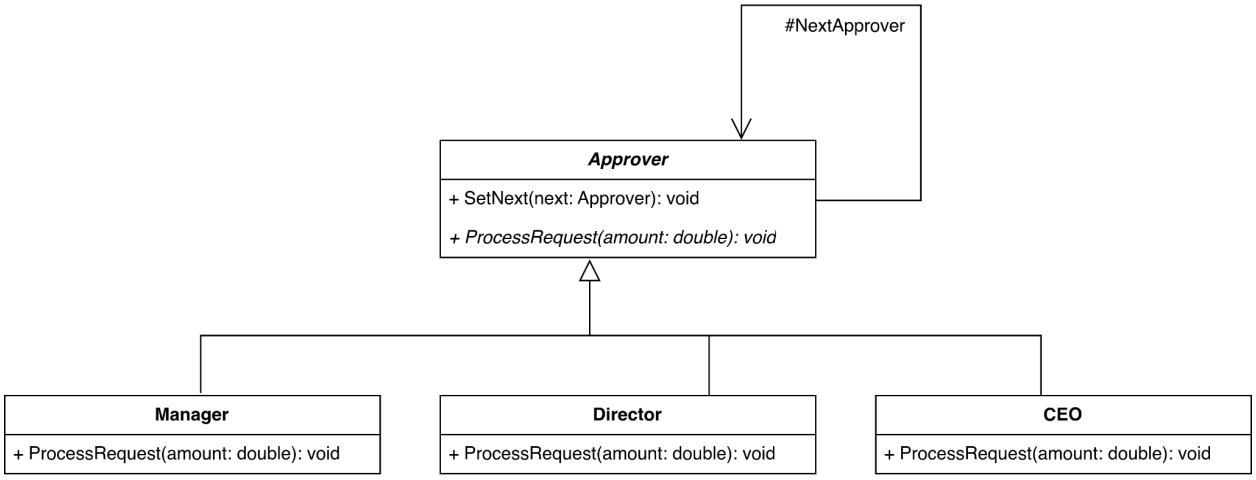
```
[{request}] unhandled.");
        }
    }
}

// Client - builds the chain and fires requests at the first Handler
public class Client
{
    private readonly Handler _firstHandler;

    public Client(Handler firstHandler) => _firstHandler = firstHandler;

    public void SendRequest(int request)
    {
        Console.WriteLine($"\nClient: sending request [{request}]");
        _firstHandler.HandleRequest(request);
    }
}

// Entry point
class Program
{
    static void Main()
    {
        // Build the chain: ConcreteHandler1 -> ConcreteHandler2
        var handler1 = new ConcreteHandler1();
        var handler2 = new ConcreteHandler2();
        handler1.SetSuccessor(handler2);   // handler1's successor is handler2

        var client = new Client(handler1);
```

```
        // Handled by ConcreteHandler1
        client.SendRequest(5);

        // Handled by ConcreteHandler2
        client.SendRequest(15);

        // Passed through both, unhandled
        client.SendRequest(25);

        // Handled by ConcreteHandler1 (boundary)
        client.SendRequest(0);

        // Handled by ConcreteHandler2 (boundary)
        client.SendRequest(10);
    }
}
```

Example

Expense Approval

ក្នុងក្រុមហ៊ុនមួយ ការសុំបើកលុយត្រូវឆ្លងកាត់ថ្នាក់ដឹកនាំតាមលំដាប់លំដោយ។ បើទឹកប្រាក់តិច (ក្រោម $100) ប្រធានក្រុម (Manager) ជាអ្នកអនុម័ត។ បើច្រើនជាងនេះ ត្រូវបញ្ជូនទៅនាយកផ្នែក (Director) ហើយបើច្រើនខ្លាំង ត្រូវបញ្ជូនទៅអគ្គនាយក (CEO)។

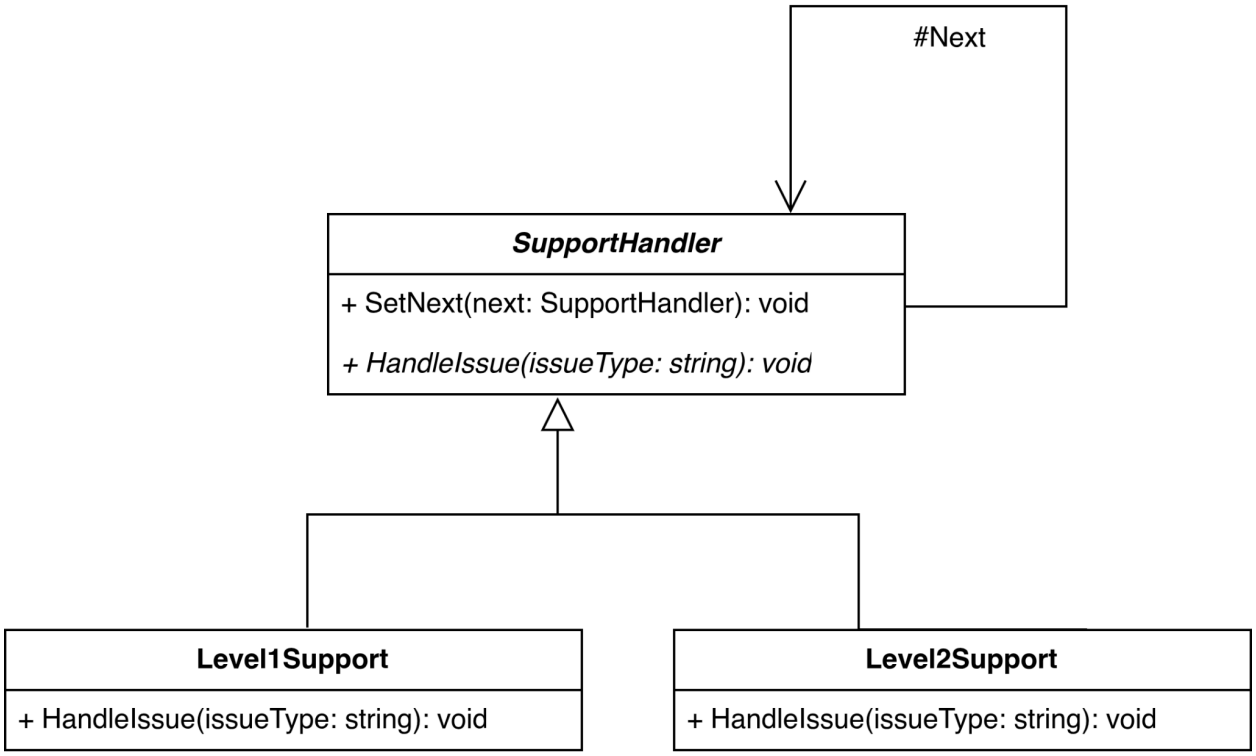



```
using System;

// 1. Handler Abstract Class
public abstract class Approver {
    protected Approver NextApprover;
    public void SetNext(Approver next) => NextApprover = next;
    public abstract void ProcessRequest(double amount);
}

// 2. Concrete Handlers
public class Manager : Approver {
    public override void ProcessRequest(double amount) {
        if (amount <= 100) Console.WriteLine($"Manager approved ${amount}");
        else NextApprover?.ProcessRequest(amount);
    }
}

public class Director : Approver {
    public override void ProcessRequest(double amount) {
        if (amount <= 1000) Console.WriteLine($"Director approved ${amount}");
        else NextApprover?.ProcessRequest(amount);
```

```
    }
}

public class CEO : Approver {
    public override void ProcessRequest(double amount) {
        Console.WriteLine($"CEO approved ${amount}");
    }
}
```

Technical Support

នៅពេលអតិថិជនខលមកសួរពីបញ្ហា បុគ្គលិកជួរមុខ (Level 1) នឹងព្យាយាមដោះស្រាយ។ បើដោះស្រាយមិនចេញ គេនឹងបញ្ជូនទៅផ្នែកបច្ចេកទេស (Level 2) ហើយបើនៅតែមិនចេញទៀត នឹងបញ្ជូនទៅអ្នកជំនាញជាន់ខ្ពស់ (Level 3)។

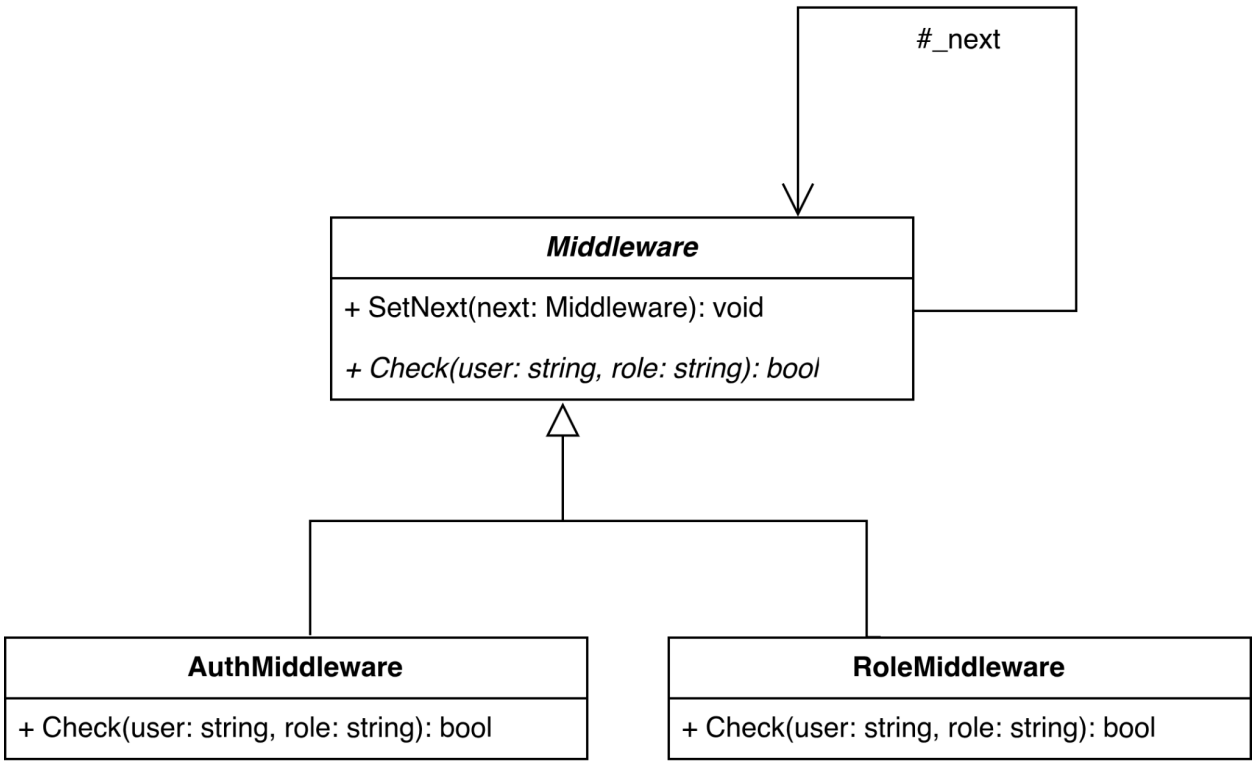



```
// 1. Handler
public abstract class SupportHandler {
```

```
    protected SupportHandler Next;
    public void SetNext(SupportHandler next) => Next = next;
    public abstract void HandleIssue(string issueType);
}

// 2. Concrete Handlers
public class Level1Support : SupportHandler {
    public override void HandleIssue(string issueType) {
        if (issueType == "Basic") Console.WriteLine("Level 1 handled the basic
issue.");
        else Next?.HandleIssue(issueType);
    }
}

public class Level2Support : SupportHandler {
    public override void HandleIssue(string issueType) {
        if (issueType == "Technical") Console.WriteLine("Level 2 handled the
technical issue.");
        else Next?.HandleIssue(issueType);
    }
}
```

Request Middleware/Validation

នៅពេលមាន Request ចូលមកក្នុង Website យើងត្រូវធ្វើការពិនិត្យតាមលំដាប់៖ ១. ពិនិត្យថាតើមាន User នេះមែនអត់ (Authentication), ២. ពិនិត្យថាគេមានសិទ្ធិចូលមើលអត់ (Authorization), ៣. ពិនិត្យថាទិន្នន័យត្រឹមត្រូវអត់ (Validation)។ បើដំណាក់កាលណាមួយមិនជាប់ គឺឈប់ត្រឹមហ្នឹងតែម្តង។




```
// 1. Handler
public abstract class Middleware {
    protected Middleware _next;
    public void SetNext(Middleware next) => _next = next;
    public abstract bool Check(string user, string role);
}

// 2. Concrete Handlers
public class AuthMiddleware : Middleware {
    public override bool Check(string user, string role) {
        if (user != "admin") {
```

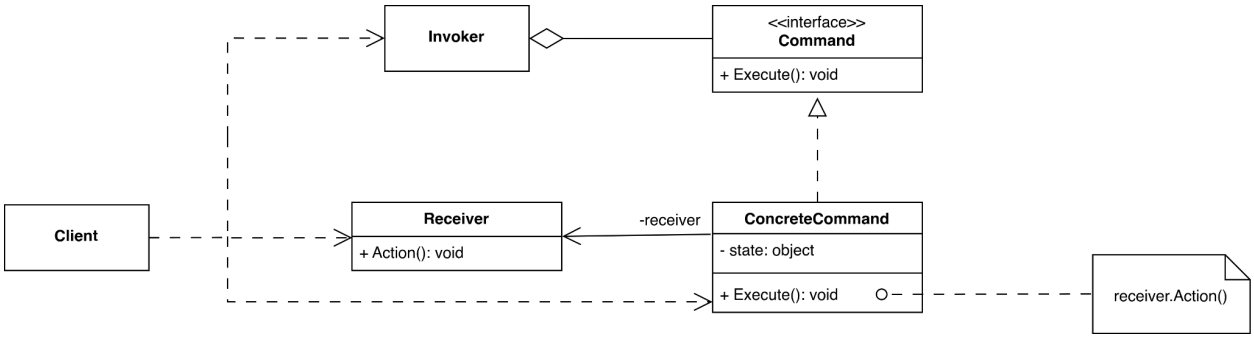
```
            Console.WriteLine("Auth Failed!");
            return false;
        }
        return _next?.Check(user, role) ?? true;
    }
}

public class RoleMiddleware : Middleware {
    public override bool Check(string user, string role) {
        if (role != "SuperAdmin") {
            Console.WriteLine("Access Denied! You are not SuperAdmin.");
            return false;
        }
        return _next?.Check(user, role) ?? true;
    }
}
```

Command Pattern

Intent

រៀបខ្ជប់សេចក្ដីថ្លែងការណ៍ (Request) ឱ្យទៅជាវត្ថុ (Object) ដែលអនុញ្ញាតឱ្យអ្នកកំណត់ប៉ារ៉ាម៉ែត្រ (Parameterize) ទៅកាន់ Client ជាមួយនឹងសេចក្ដីថ្លែងការណ៍ផ្សេងៗគ្នា រៀបជាជួរ (Queue) ឬកត់ត្រាទុក (Log) និងគាំទ្រដល់ប្រតិបត្តិការដែលអាចត្រឡប់ថយក្រោយវិញបាន (Undoable operations)។

UML Class Diagram



- Command
  - declares an interface for executing an operation.
- ConcreteCommand
  - defines a binding between a Receiver object and an action.
  - implements Execute by invoking the corresponding operation(s) on Receiver.
- Client
  - creates a ConcreteCommand object and sets its receiver.
- Invoker
  - asks the command to carry out the request.
- Receiver
  - knows how to perform the operations associated with carrying out a request. Any class may serve as a Receiver.

Sample Code

```
using System;
using System.Collections.Generic;

// Command interface
public interface ICommand
{
    void Execute();
}

// Receiver - knows how to perform the actual work
public class Receiver
{
    public string Name { get; }

    public Receiver(string name) => Name = name;

    public void Action() =>
        Console.WriteLine($"Receiver [{Name}]: Action() performed.");
}

// ConcreteCommand - implements ICommand, binds a Receiver to an action
public class ConcreteCommand : ICommand
{
    private readonly Receiver _receiver;   // -receiver association
    private object _state;                 // - state: object

    public ConcreteCommand(Receiver receiver, object state)
    {
        _receiver = receiver;
```

```
        _state     = state;
    }

    // Execute note: receiver.Action()
    public void Execute()
    {
        Console.WriteLine($"ConcreteCommand: Execute() -- state = [{_state}]");
        _receiver.Action();
    }
}

// Invoker - holds a Command (aggregation) and triggers it
public class Invoker
{
    private readonly List<ICommand> _commandQueue = new();

    // Store a command to be invoked later
    public void StoreCommand(ICommand command)
    {
        Console.WriteLine("Invoker: command stored.");
        _commandQueue.Add(command);
    }

    // Fire all stored commands
    public void ExecuteCommands()
    {
        Console.WriteLine("Invoker: executing all stored commands...");
        foreach (var command in _commandQueue)
            command.Execute();
        _commandQueue.Clear();
    }
```

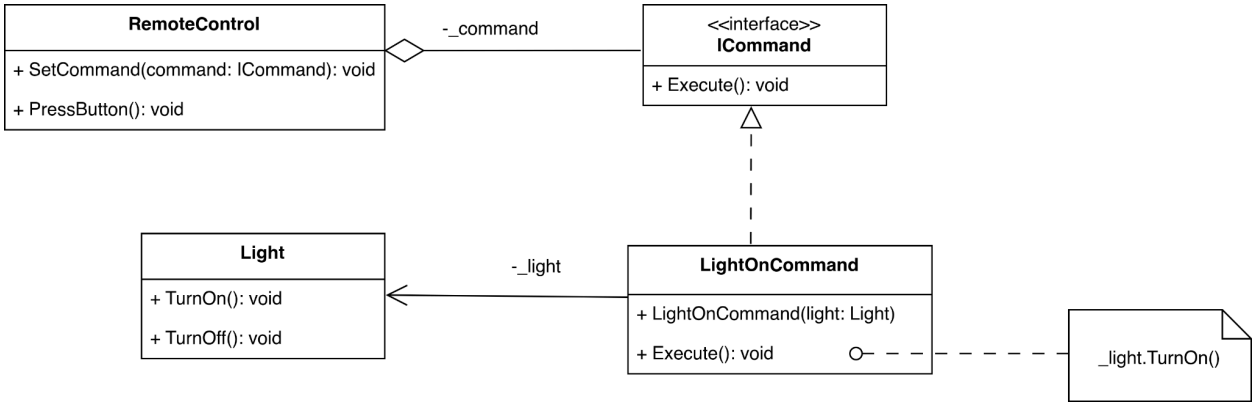
```
    // Fire a single command immediately
    public void ExecuteCommand(ICommand command)
    {
        Console.WriteLine("Invoker: executing single command immediately...");
        command.Execute();
    }
}

// Client - creates Receiver, ConcreteCommand, and wires them to the Invoker
public class Client
{
    public void Run()
    {
        // Create receivers
        var receiver1 = new Receiver("ReceiverA");
        var receiver2 = new Receiver("ReceiverB");

        // Create concrete commands and bind them to receivers
        ICommand command1 = new ConcreteCommand(receiver1, "state-alpha");
        ICommand command2 = new ConcreteCommand(receiver2, "state-beta");
        ICommand command3 = new ConcreteCommand(receiver1,
"state-gamma");

        // Create invoker and store commands
        var invoker = new Invoker();

        Console.WriteLine("=== Storing commands in Invoker ===");
        invoker.StoreCommand(command1);
        invoker.StoreCommand(command2);
        invoker.StoreCommand(command3);
```

```
        Console.WriteLine();
        Console.WriteLine("=== Invoker executing all stored commands ===");
        invoker.ExecuteCommands();

        Console.WriteLine();
        Console.WriteLine("=== Invoker executing a single command
immediately ===");
        ICommand command4 = new ConcreteCommand(receiver2, "state-delta");
        invoker.ExecuteCommand(command4);
    }
}

// Entry point
class Program
{
    static void Main()
    {
        var client = new Client();
        client.Run();
    }
}
```

Example

Smart Home Remote

អ្នកមានតេឡេបញ្ជាមួយដែលមានប៊ូតុងច្រើន។ អ្នកចង់ឱ្យប៊ូតុងនីមួយៗអាចបញ្ជាឧបករណ៍ខុសៗគ្នា (ដូចជាបើកភ្លើង ឬបើកកង្ហារ)។ បើអ្នកសរសេរកូដភ្ជាប់ប៊ូតុងទៅនឹងឧបករណ៍ផ្ទាល់
វានឹងពិបាកផ្លាស់ប្តូរមុខងារប៊ូតុងនៅពេលក្រោយ។

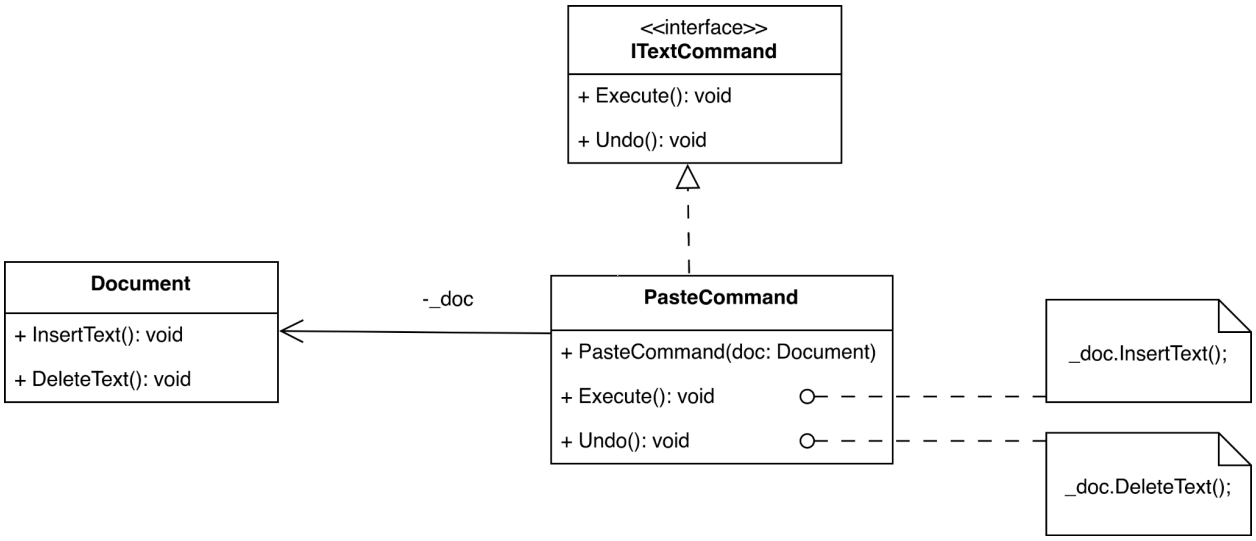



```
using System;

// 1. Command Interface
public interface ICommand {
    void Execute();
}

// 2. Receiver (អ្នកដែលត្រូវអនុវត្តការងារពិតប្រាកដ)
public class Light {
    public void TurnOn() => Console.WriteLine("The light is ON");
    public void TurnOff() => Console.WriteLine("The light is OFF");
}

// 3. Concrete Command
public class LightOnCommand : ICommand {
    private Light _light;
    public LightOnCommand(Light light) => _light = light;
    public void Execute() => _light.TurnOn();
}

// 4. Invoker (អ្នកកាន់ប៊ូតុង)
public class RemoteControl {
    private ICommand _command;
```

```
    public void SetCommand(ICommand command) => _command = command;
    public void PressButton() => _command.Execute();
}
```

Text Editor Undo/Redo

នៅក្នុងកម្មវិធីសរសេរអត្ថបទ រាល់ពេលដែលអ្នកចុច "Cut", "Copy" ឬ "Paste"
អ្នកចង់ឱ្យកម្មវិធីចងចាំសកម្មភាពទាំងនោះ ដើម្បីអាចធ្វើការ Undo (ត្រឡប់ក្រោយ) វិញបាន។

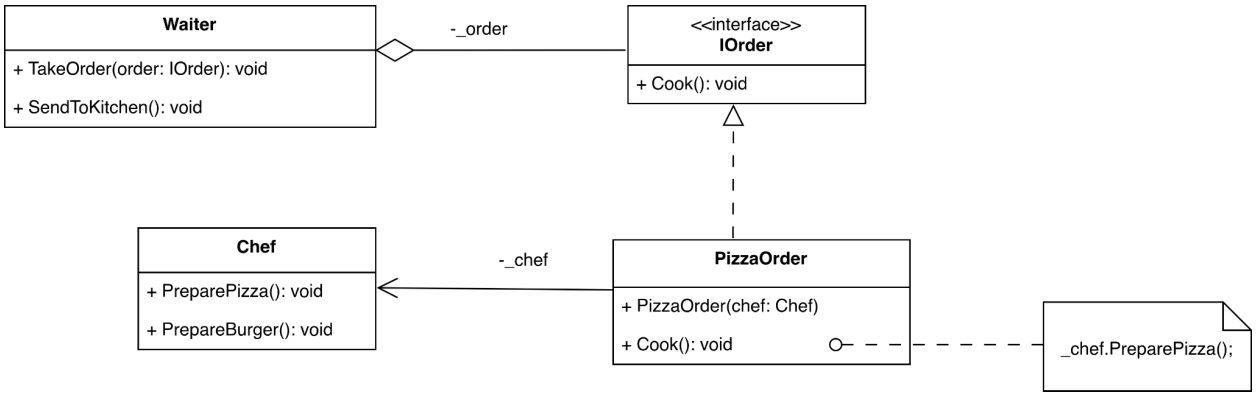



```
// 1. Command Interface
public interface ITextCommand {
    void Execute();
    void Undo();
}

// 2. Receiver
public class Document {
    public void InsertText() => Console.WriteLine("Text inserted.");
    public void DeleteText() => Console.WriteLine("Text deleted.");
}

// 3. Concrete Command
```

```
public class PasteCommand : ITextCommand {
    private Document _doc;
    public PasteCommand(Document doc) => _doc = doc;

    public void Execute() => _doc.InsertText();
    public void Undo() => _doc.DeleteText();
}
```

Restaurant Ordering

នៅពេលអ្នកទៅញ៉ាំអី អ្នកសរសេរម្ហូបដែលចង់បានដាក់ក្នុង "ក្រដាសកម្មង់"។ ក្រដាសនោះគឺជា Command Object។ អ្នករត់តុ (Invoker) យកក្រដាសនោះទៅឱ្យចុងភៅ (Receiver)។ ចុងភៅមិនចាំបាច់ដឹងថាអ្នកណាជាអ្នកកម្មង់ទេ គាត់គ្រាន់តែធ្វើតាមអ្វីដែលសរសេរក្នុងក្រដាសនោះ។




```
// 1. Command
public interface IOrder {
    void Cook();
}

// 2. Receiver
public class Chef {
    public void PreparePizza() => Console.WriteLine("Chef is cooking Pizza...");
    public void PrepareBurger() => Console.WriteLine("Chef is cooking
Burger...");
}
```

```
// 3. Concrete Command
public class PizzaOrder : IOrder {
    private Chef _chef;
    public PizzaOrder(Chef chef) => _chef = chef;
    public void Cook() => _chef.PreparePizza();
}

// 4. Invoker
public class Waiter {
    private IOrder _order;
    public void TakeOrder(IOrder order) => _order = order;
    public void SendToKitchen() => _order.Cook();
}
```